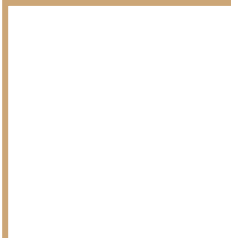


Data Structures

Mushtari Sadia

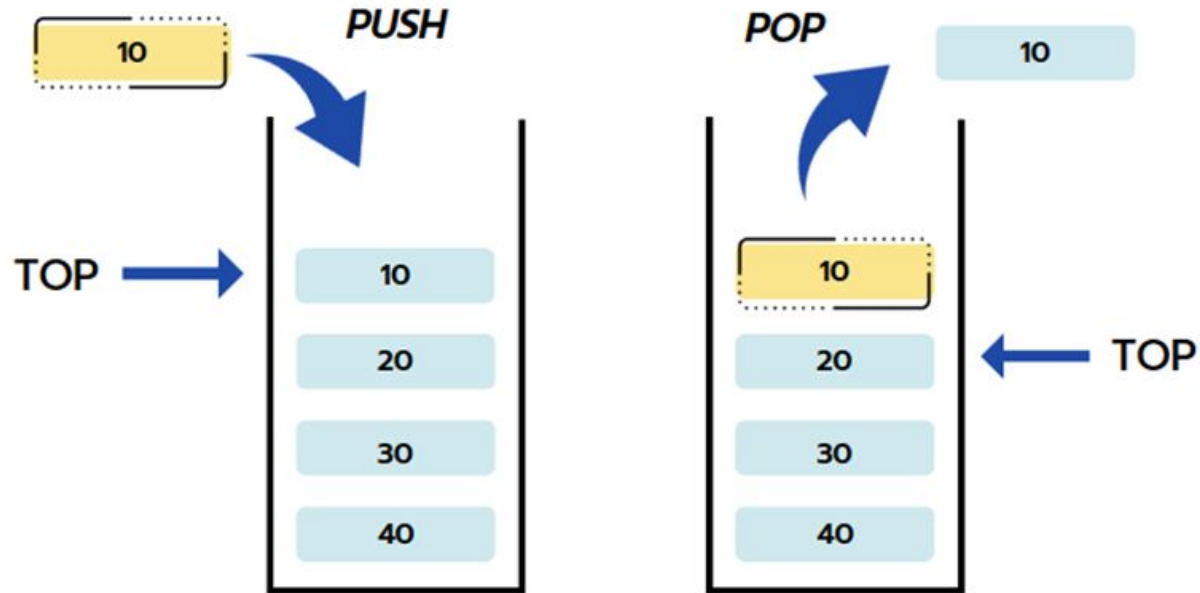




Stack



Introduction

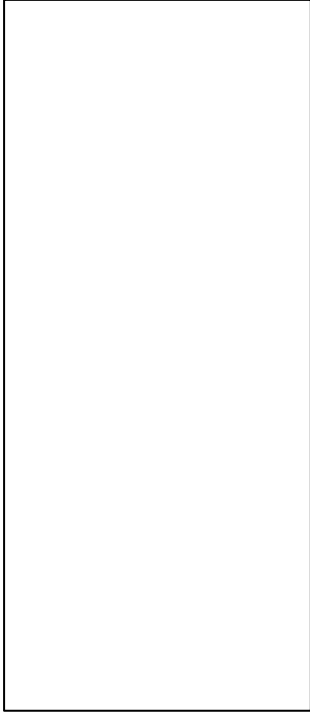


STACK DATA STRUCTURE

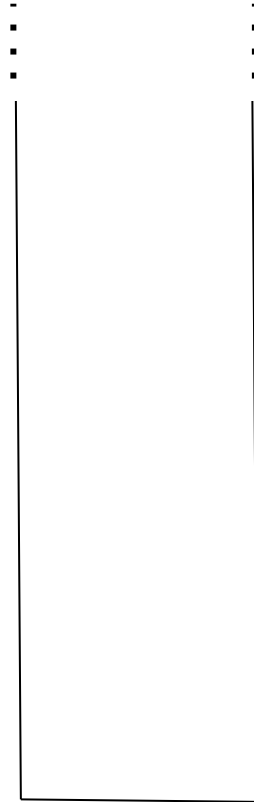
Basic Operations

- **Push(obj)**: adds obj to the top of the stack ("overflow" error if the stack has fixed capacity, and is full)
- **Pop**: removes and returns the item from the top of the stack ("underflow" error if the stack is empty)
- **Peek**: returns the item that is on the top of the stack, but does not remove it ("underflow" error if the stack is empty)

Stack



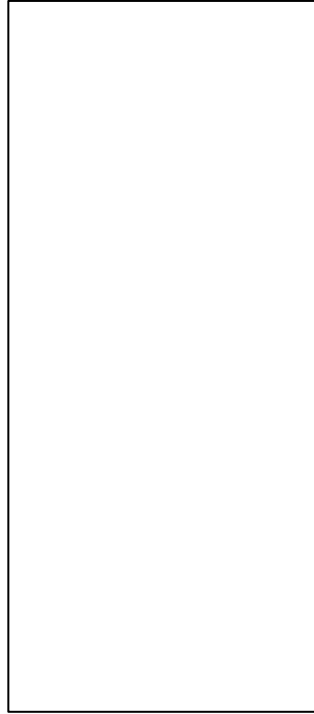
Fixed capacity - **implemented with array**



Unlimited capacity - **implemented with linked list**

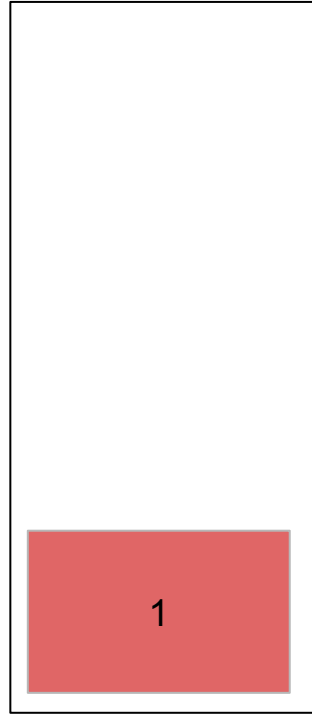
Fixed Capacity Stack Example

Create
Stack of
Size 4

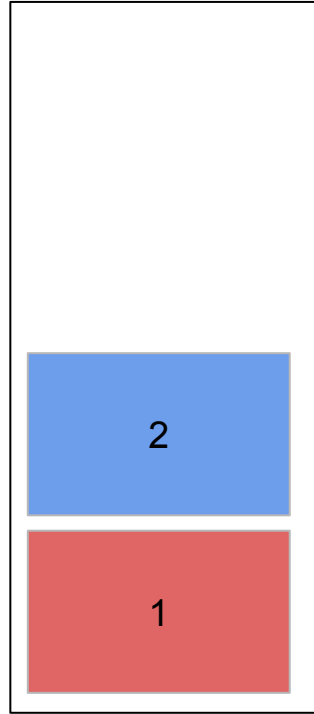


(Implemented with array)

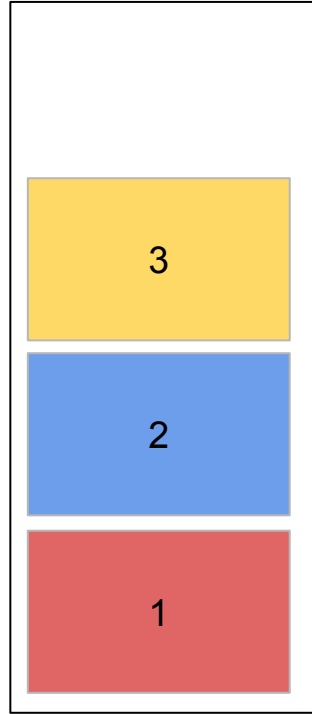
Push(1)



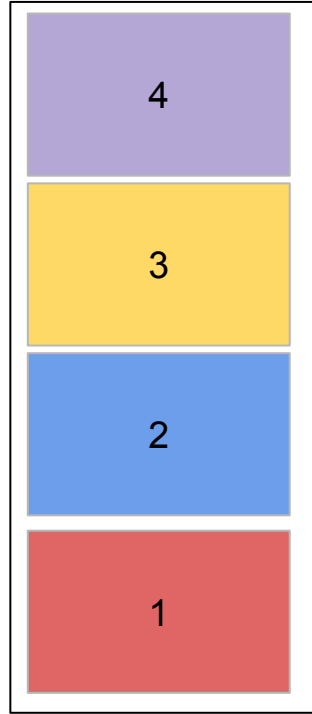
Push(2)



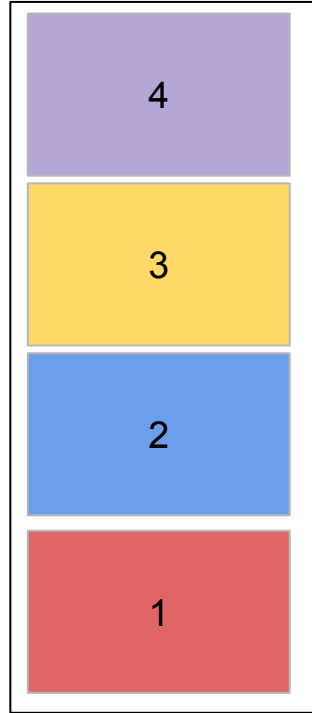
Push(3)



Push(4)

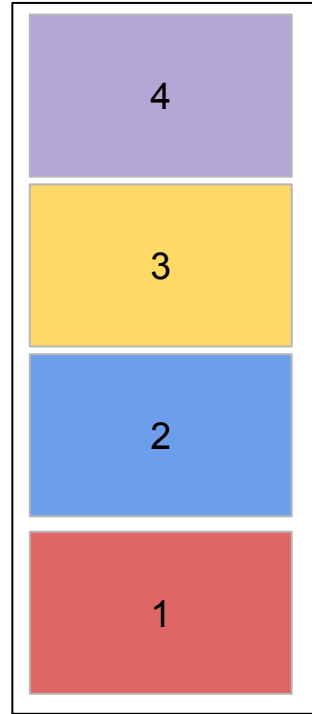


Push(5)



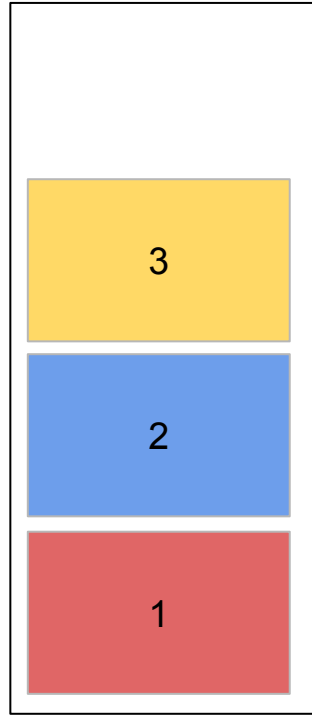
Stack Overflow Error!

Peek



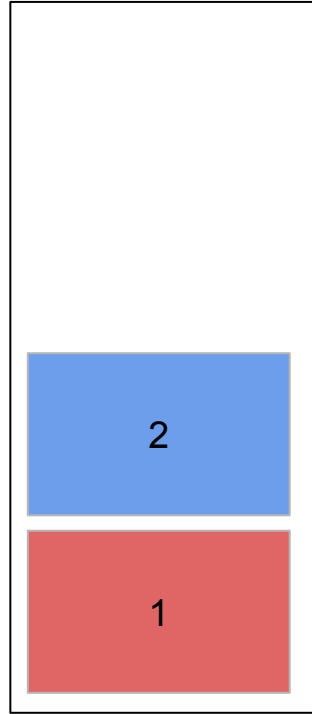
Will return 4, but will not
remove 4 from the stack

Pop

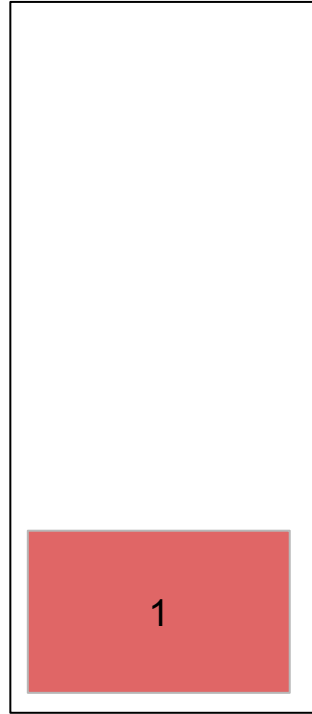


Will return 4 and remove
4 from the stack

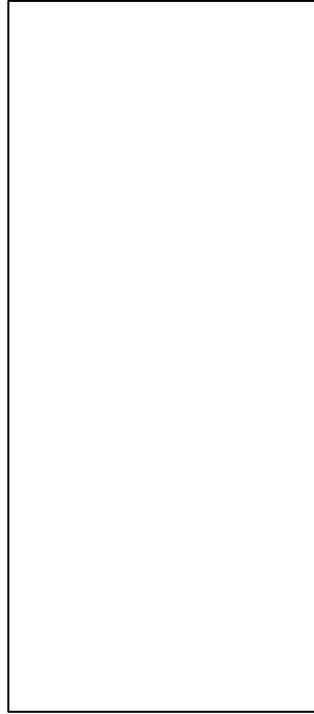
Pop



Pop

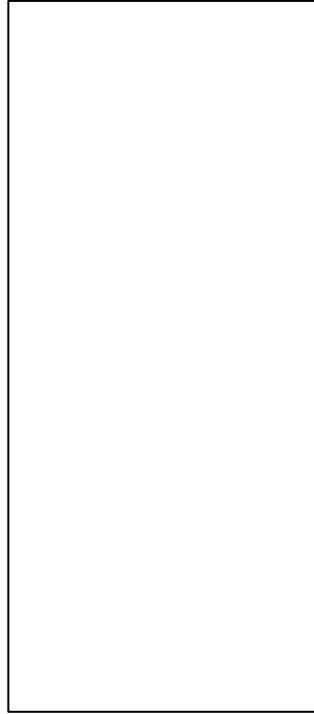


Pop



Pop

Stack Underflow Error!



```
import numpy as np

class Stack:
    def __init__(self):
        self.stack = np.zeros(4, dtype=int)
        self.top = -1

    def push(self, item):
        if self.top == len(self.stack) - 1:
            print("Stack overflow")
        else:
            self.top += 1
            self.stack[self.top] = item

    def pop(self):
        if self.top == -1:
            print("Stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return item
```

Code

Implementing The Structure with Array

```
def peek(self):
    if self.top == -1:
        print("Stack is empty")
    else:
        return self.stack[self.top]
```

```
stack = Stack()  
stack.push(1)  
stack.push(2)  
stack.push(3)  
stack.push(4)  
stack.push(5)  
print(stack.peek())  
print(stack.pop())  
stack.pop()  
print(stack.pop())  
stack.pop()  
stack.pop()
```

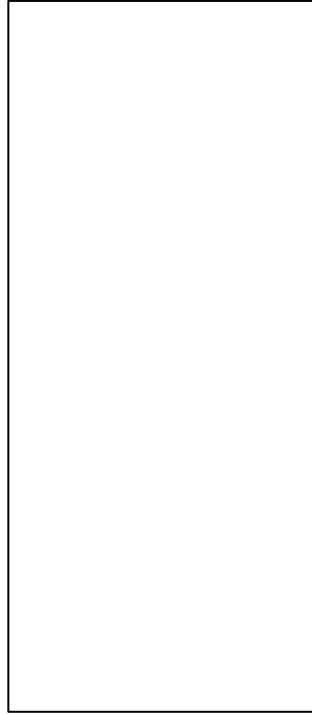
Driver Code

Using the
implemented
structure

Fixed Capacity Stack Example

Create
Stack of
Size 4

top = -1



Driver code:

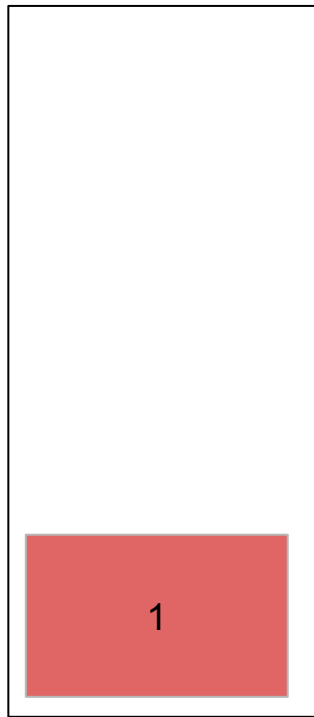
```
stack = Stack()
```

Code running
behind the scene:

```
def __init__(self):  
    self.stack = np.zeros(4, dtype=int)  
    self.top = -1
```

Push(1)

top = 0



Driver code:

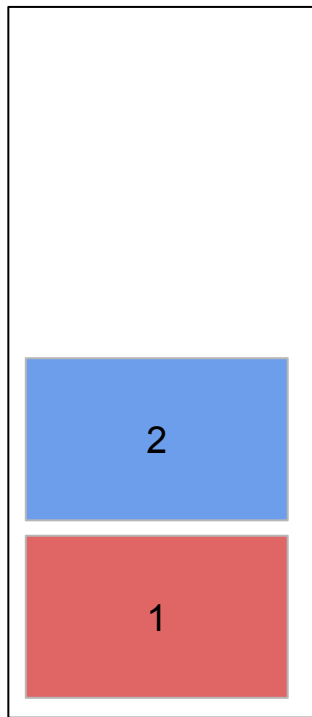
```
stack.push(1)
```

Code running
behind the scene:

```
def push(self, item):  
    if self.top == len(self.stack) - 1:  
        print("Stack overflow")  
    else:  
        self.top += 1  
        self.stack[self.top] = item
```

Push(2)

top = 1



Driver code:

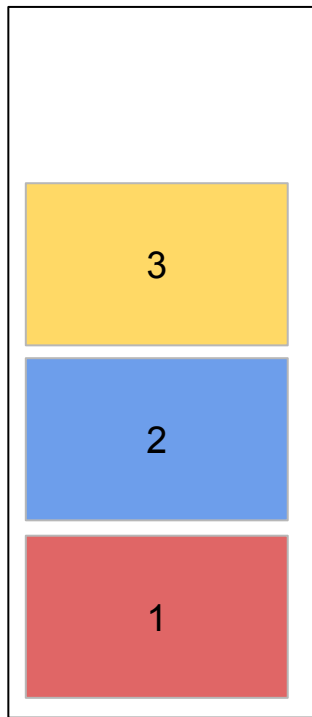
```
stack.push(2)
```

Code running
behind the scene:

```
def push(self, item):  
    if self.top == len(self.stack) - 1:  
        print("Stack overflow")  
    else:  
        self.top += 1  
        self.stack[self.top] = item
```

Push(3)

top = 2



Driver code:

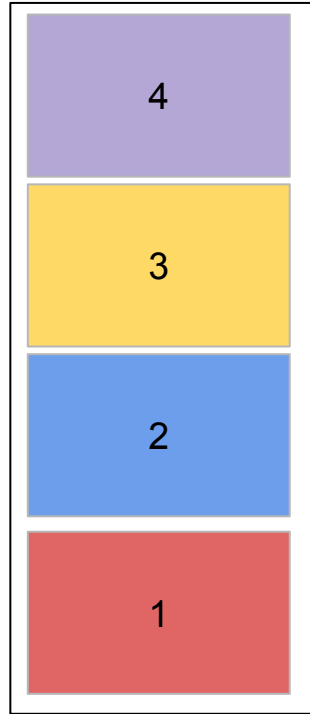
```
stack.push(3)
```

Code running
behind the scene:

```
def push(self, item):  
    if self.top == len(self.stack) - 1:  
        print("Stack overflow")  
    else:  
        self.top += 1  
        self.stack[self.top] = item
```

Push(4)

top = 3



Driver code:

```
stack.push(4)
```

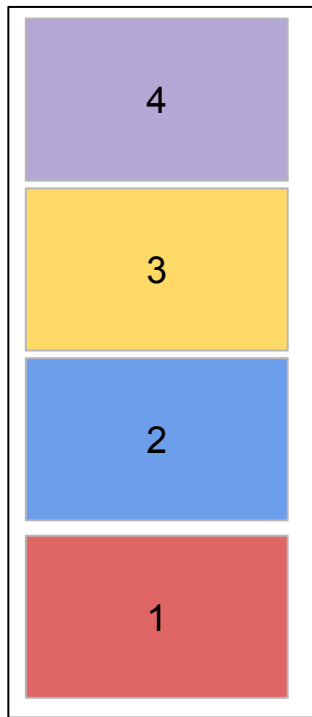
Code running
behind the scene:

```
def push(self, item):  
    if self.top == len(self.stack) - 1:  
        print("Stack overflow")  
    else:  
        self.top += 1  
        self.stack[self.top] = item
```


Push(5)

Stack Overflow Error!

top = 3



Driver code:

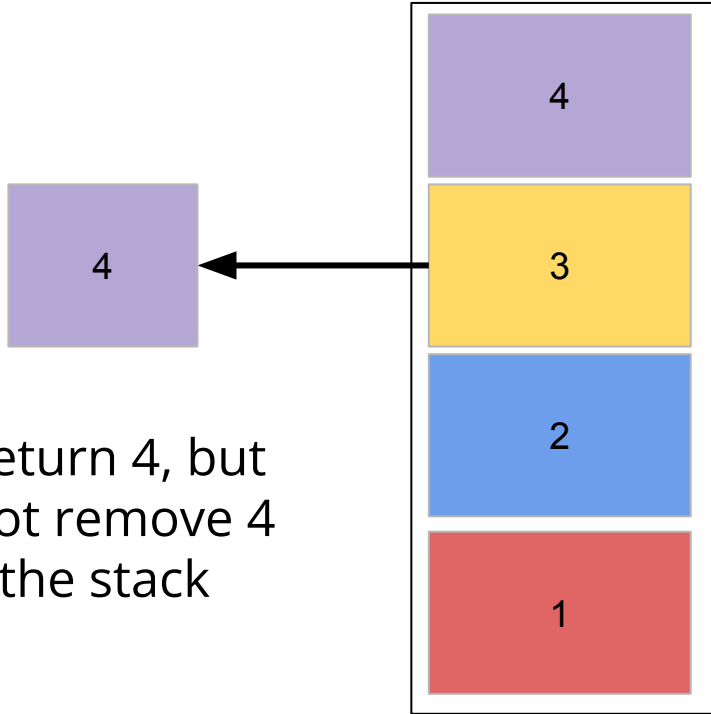
```
stack.push(5)
```

Code running
behind the scene:

```
def push(self, item):  
    if self.top == len(self.stack) - 1:  
        print("Stack overflow")  
    else:  
        self.top += 1  
        self.stack[self.top] = item
```

Peek

top = 3



Will return 4, but
will not remove 4
from the stack

Driver code:

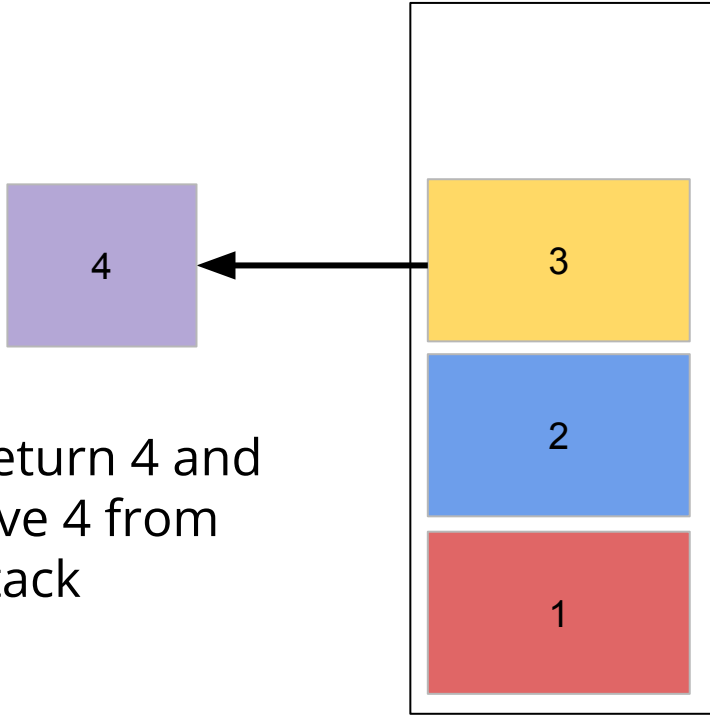
```
print(stack.peek())
```

Code running
behind the scene:

```
def peek(self):  
    if self.top == -1:  
        print("Stack is empty")  
    else:  
        return  
self.stack[self.top]
```

Pop

top = 2



Will return 4 and
remove 4 from
the stack

Driver code:

```
print(stack.pop())
```

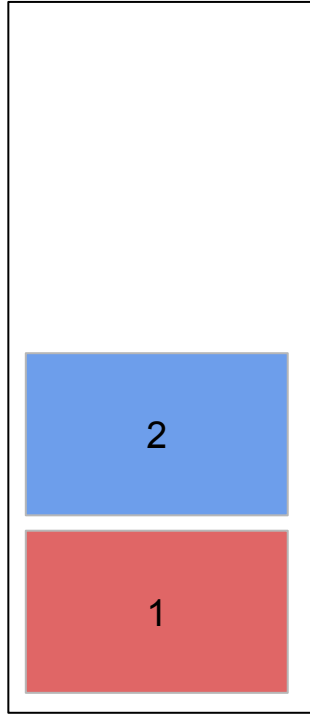
Code running
behind the scene:

```
def pop(self):  
    if self.top == -1:  
        print("Stack underflow")  
    else:  
        item = self.stack[self.top]  
        self.stack[self.top] = None  
        self.top -= 1  
        return item
```

Pop

Will remove 3
from the stack

top = 1



Driver code:

```
stack.pop()
```

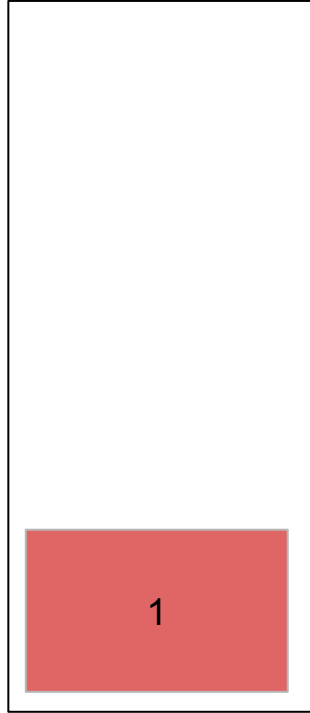
Code running
behind the scene:

```
def pop(self):  
    if self.top == -1:  
        print("Stack underflow")  
    else:  
        item = self.stack[self.top]  
        self.stack[self.top] = None  
        self.top -= 1  
        return item
```

Pop

Will remove 2
from the stack

top = 0



Driver code:

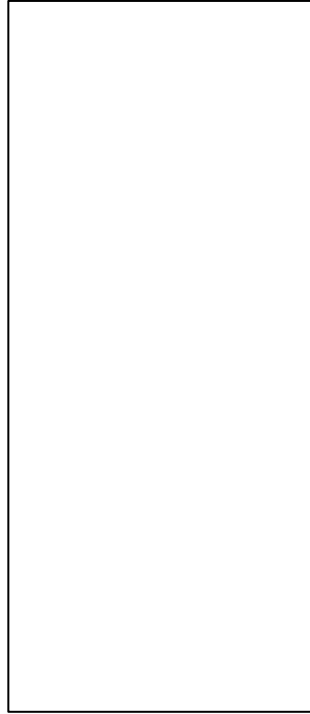
```
stack.pop()
```

Code running
behind the scene:

```
def pop(self):  
    if self.top == -1:  
        print("Stack underflow")  
    else:  
        item = self.stack[self.top]  
        self.stack[self.top] = None  
        self.top -= 1  
        return item
```

Pop

top = -1



Will remove 1
from the stack

Driver code:

```
stack.pop()
```

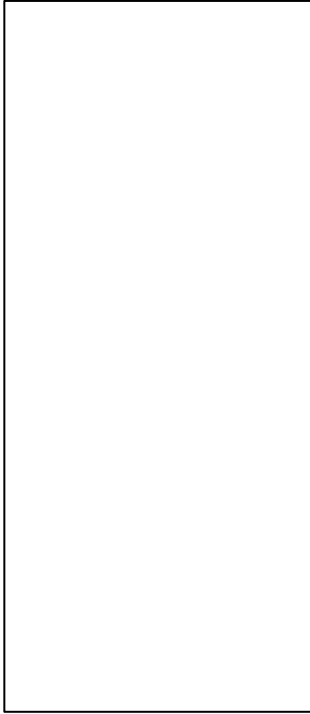
Code running
behind the scene:

```
def pop(self):  
    if self.top == -1:  
        print("Stack underflow")  
    else:  
        item = self.stack[self.top]  
        self.stack[self.top] = None  
        self.top -= 1  
        return item
```

Pop

**Stack Underflow
Error!**

top = -1



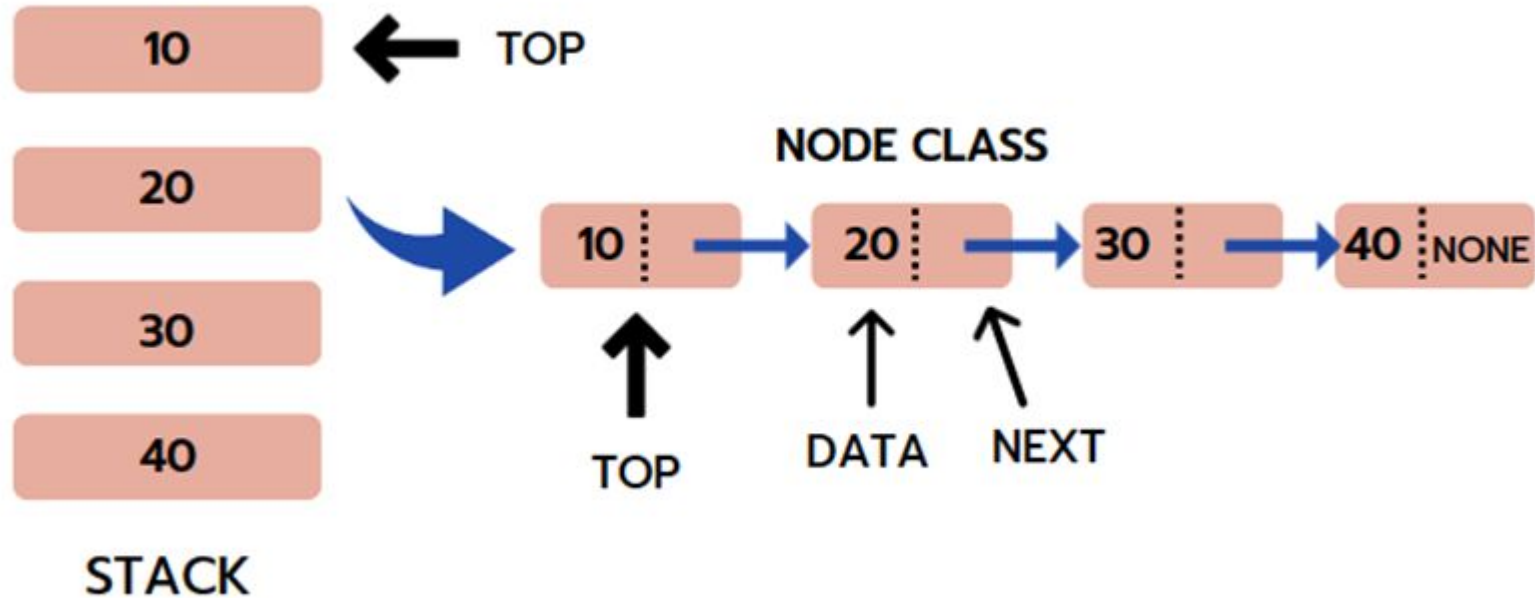
Driver code:

```
stack.pop()
```

Code running
behind the scene:

```
def pop(self):  
    if self.top == -1:  
        print("Stack underflow")  
    else:  
        item = self.stack[self.top]  
        self.stack[self.top] = None  
        self.top -= 1  
        return item
```

Linked List Based Implementation



Stack Creation

```
class Node:  
    def __init__(self,element,next):  
        self.element = element  
        self.next = next
```

Push

```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, elem):
        if self.top == None: #Stack is empty
            self.top = Node(elem, None)
        else:
            newNode = Node(elem, None)
            newNode.next = self.top
            self.top = newNode
```

Pop

```
def pop(self):  
    if self.top == None:  
        return None #Stack Underflow  
    else:  
        popped = self.top  
        self.top = self.top.next  
        popped.next = None  
        return popped.element
```

Peek

```
def peek(self):  
    if self.top == None:  
        return None #Stack Underflow  
    else:  
        return self.top.element
```

Stack Applications

1. Call Stacks

```
def main:
```

```
    A()
```

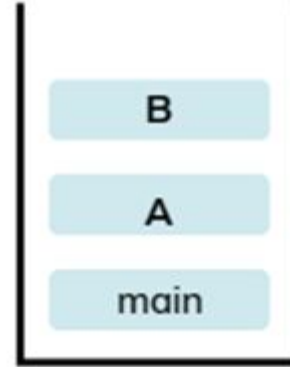
```
def A:
```

```
    B()
```

```
def B:
```

```
    print("hello world")
```

```
main()
```

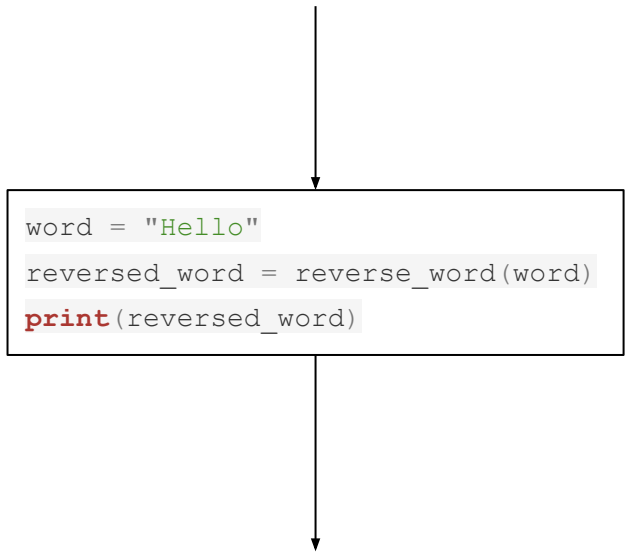


Stack Applications

2. Reverse a word

```
def reverse_word(word):  
    stack = Stack()  
  
    # Push each character of the word onto the stack  
    for char in word:  
        stack.push(char)  
  
    reversed_word = ""  
  
    # Pop each character from the stack to get the reversed word  
    while not stack.is_empty():  
        reversed_word += stack.pop()  
  
    return reversed_word
```

Hello



```
word = "Hello"  
reversed_word = reverse_word(word)  
print(reversed_word)
```

olleH

Stack Applications

3. Undo
4. Browser back button

Stack Applications

- 5. Postfix Expression Evaluation
- 6. Parenthesis Matching

Postfix Expression Evaluation

- To understand this problem, first we need to know, what is an expression tree? And what is postfix expression?

Expression Tree

- This is an expression:

$$(5 + 2) \times (3 - 1)$$

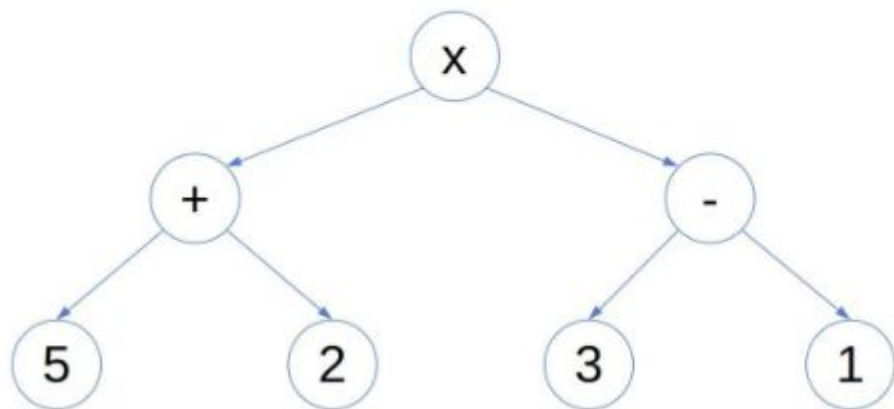
Expression Tree

$$(5 + 2) \times (3 - 1)$$

An **expression tree** is a graphical representation of an expression where:

- leaf nodes denote constant values or variables
- internal nodes contain operators

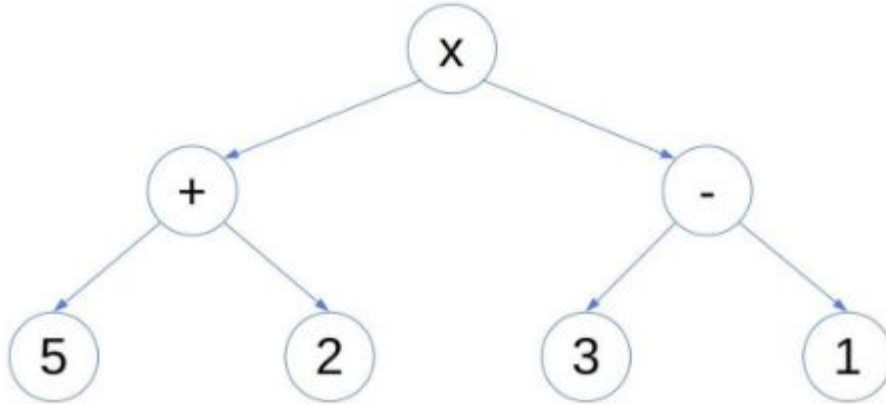
For example, here's the above expression's tree:



Prefix Traversal

pre

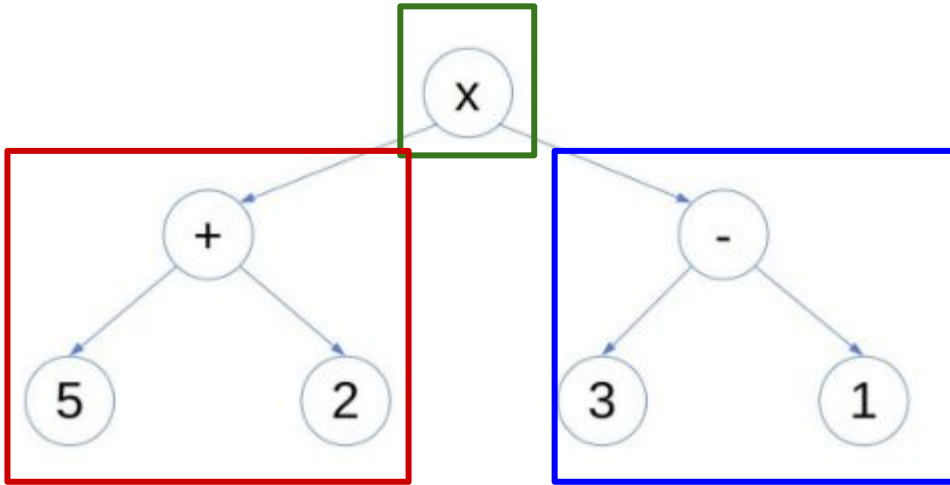
root , left, right



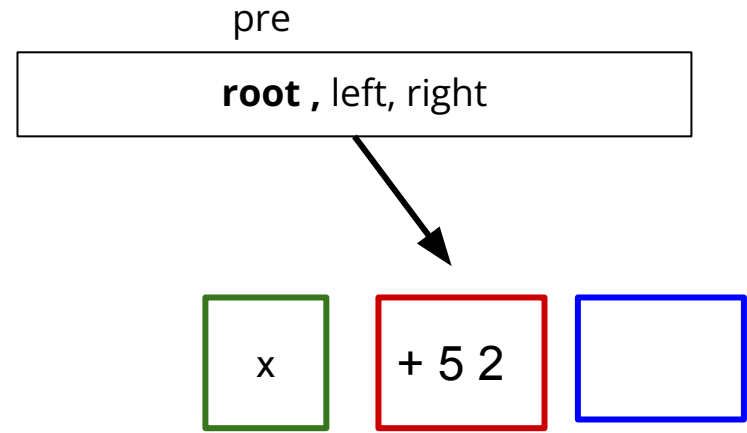
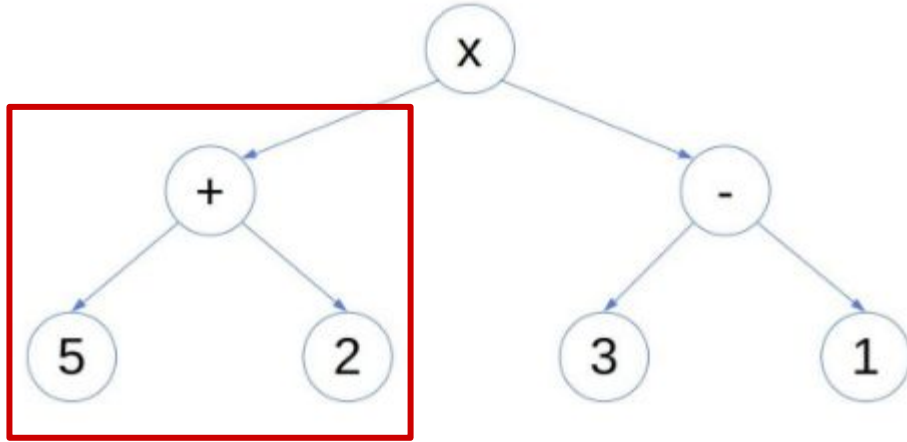
Prefix Traversal

pre

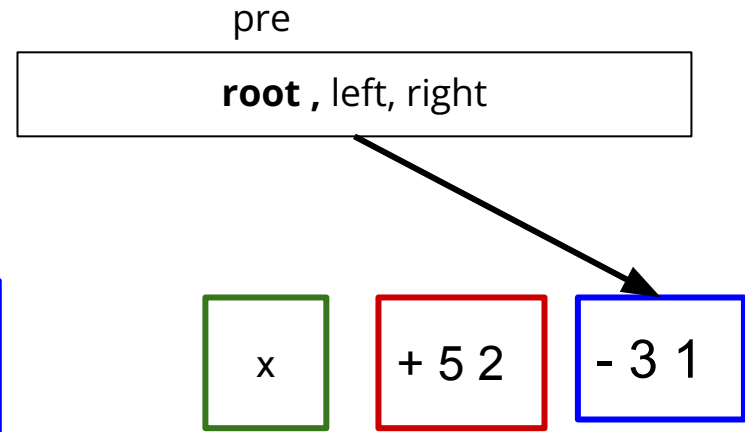
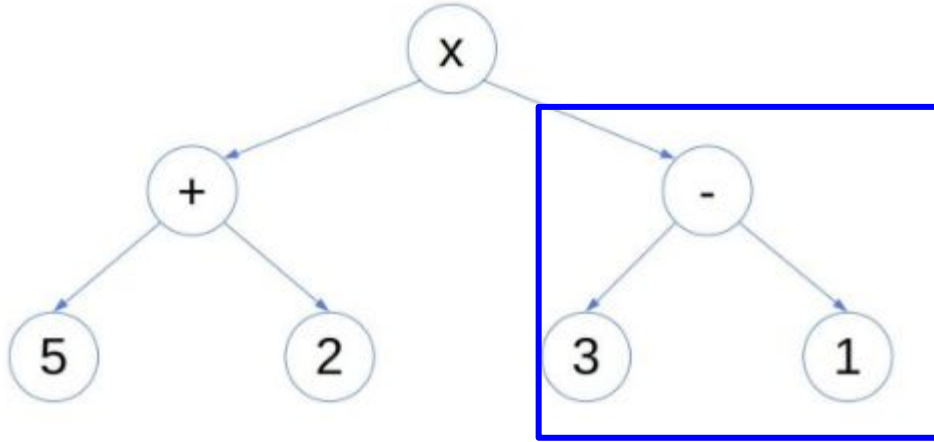
root , left, right



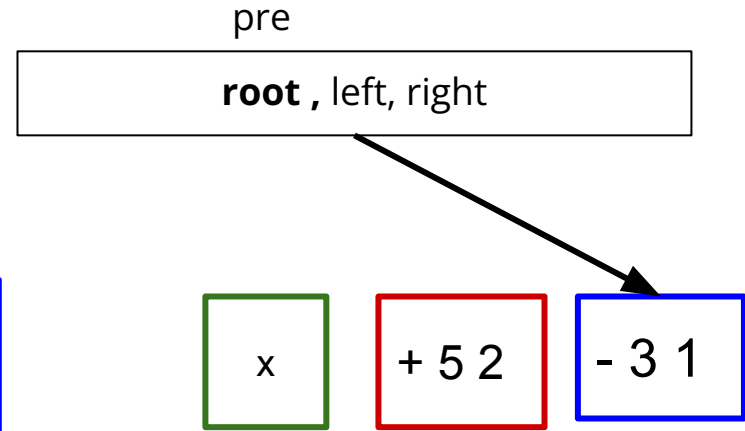
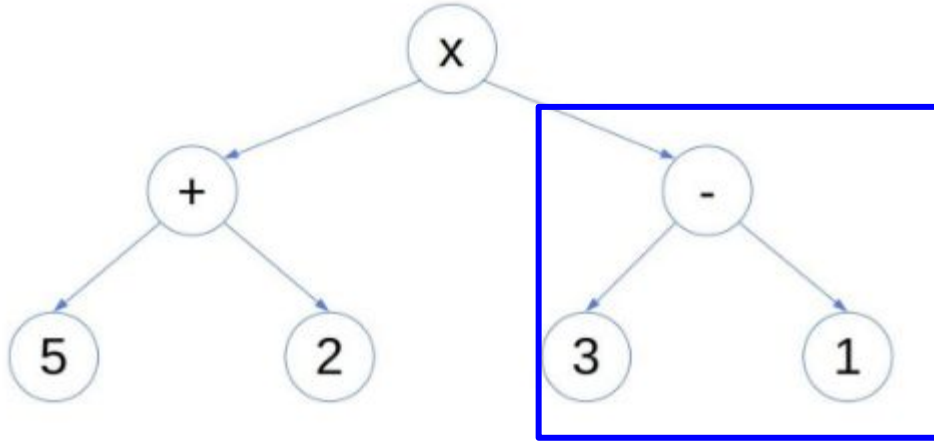
Prefix Traversal



Prefix Traversal



Prefix Traversal

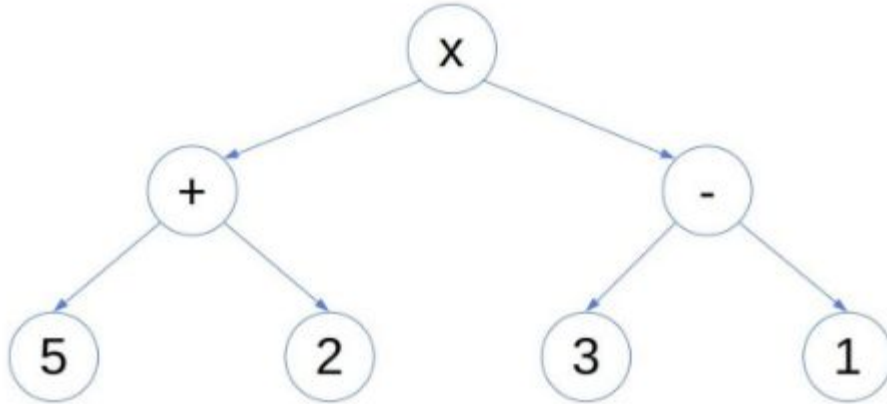


Ans: x + 5 2 - 3 1

Infix Traversal

in

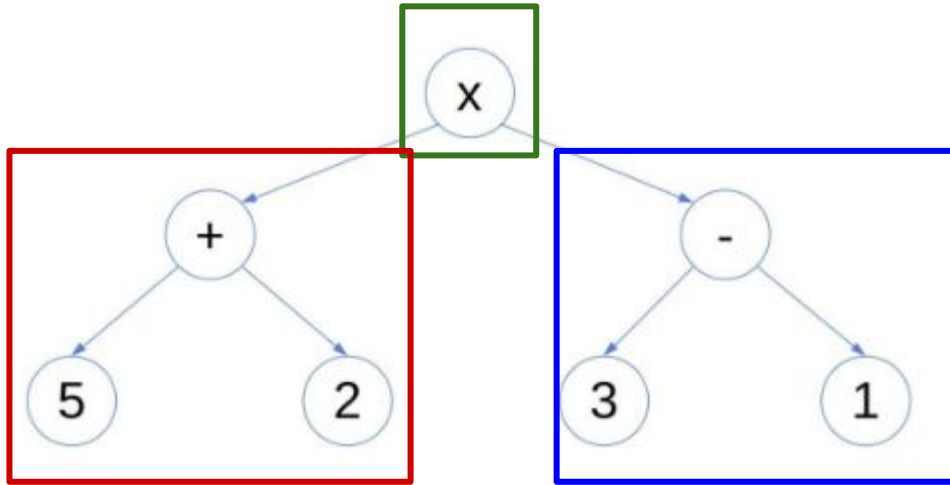
left, **root**, right



Infix Traversal

in

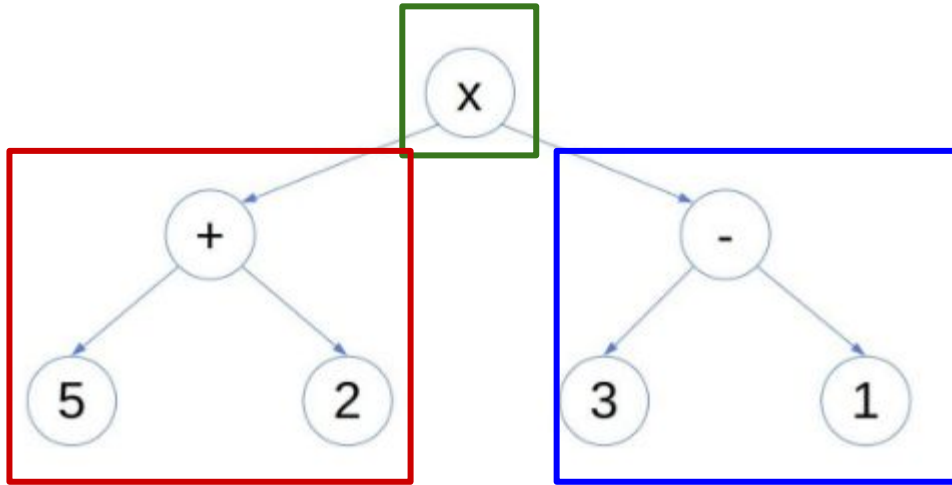
left, **root**, right



Infix Traversal

in

left, **root**, right

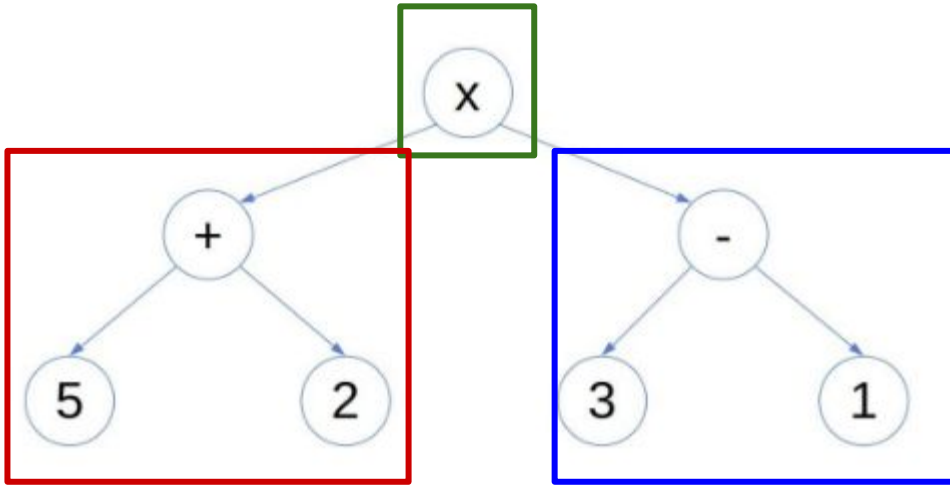


5 + 2

Infix Traversal

in

left, **root**, right



5 + 2

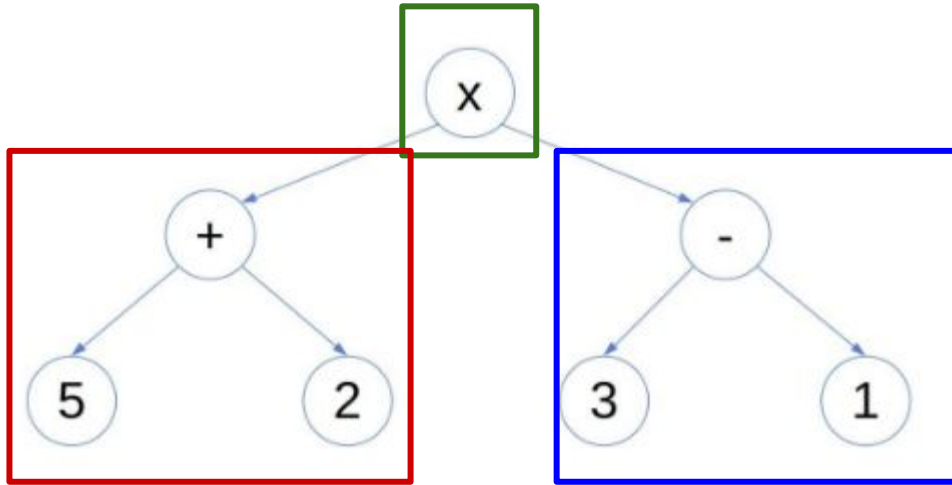
x



Infix Traversal

in

left, **root**, right



5 + 2

x

3 - 1

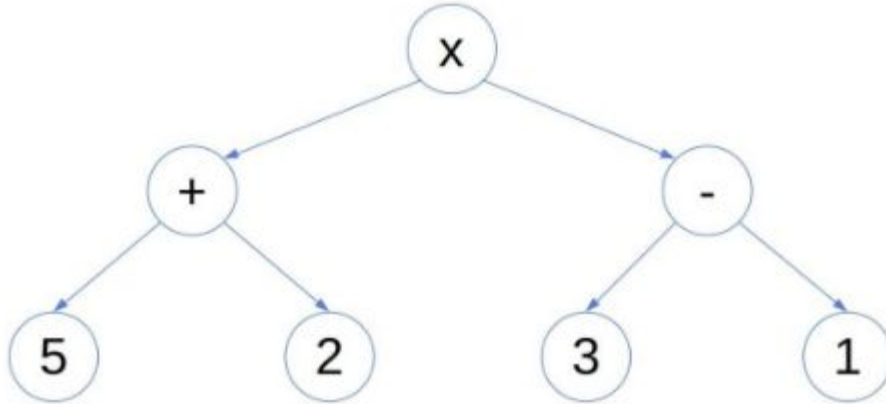
Same as this:

$$(5 + 2) \times (3 - 1)$$

Postfix Traversal

post

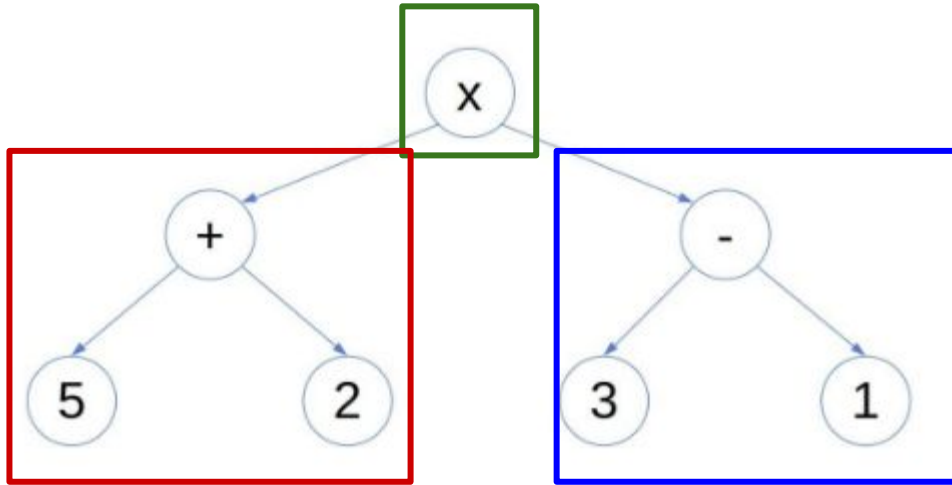
left, right, **root**



Postfix Traversal

post

left, right, **root**



5 2 +

3 1 -

x

Ans: 5 2 + 3 1 - x

Postfix Expression Evaluation

5 2 + 3 1 - ×

- Back to stack applications.
Given a postfix expression, like above, a stack helps us evaluate (calculate and find the value) of that expression.

Postfix Expression Evaluation

Input: 5 2 + 3 1 - ×

your code

Output: 14

Explanation: $(5+2) \times (3-1) = 14$, right?

Postfix Expression Evaluation

Possible solution:

Take characters one by one from the beginning

- If character is an operand (5 / 2 / 3)

 - Push to stack

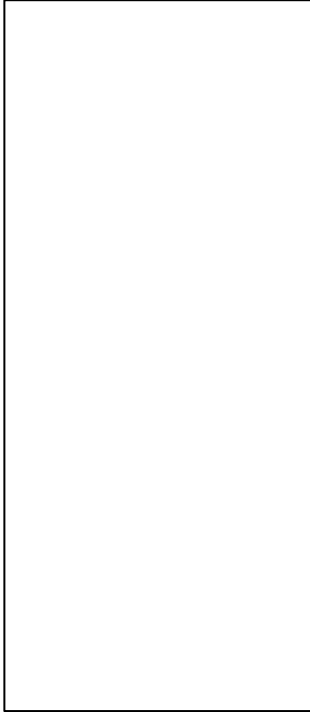
- If character is an operator (+ * / -)

 - Pop last two characters from stack

 - Perform operation on last two characters

 - Push answer to stack

Postfix Expression Evaluation



5 2 + 3 1- x
—

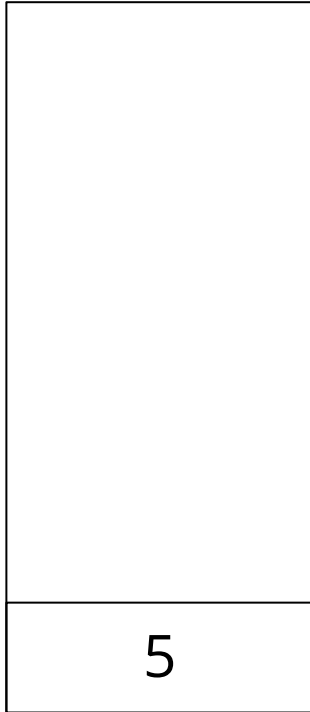
Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

Postfix Expression Evaluation



5 2 + 3 1 - x

5

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

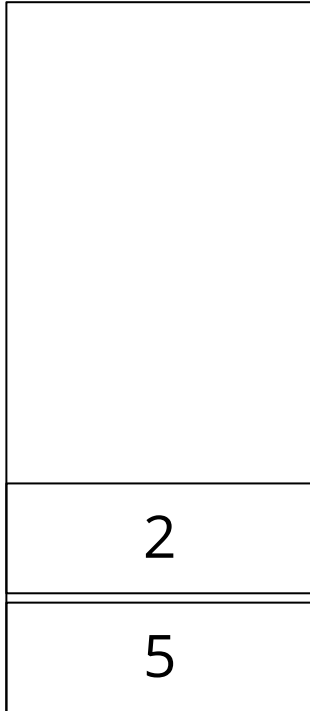
If character is an operator

Pop last two characters from stack

Perform operation on last two characters

Push answer to stack

Postfix Expression Evaluation



5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator

Pop last two characters from stack

Perform operation on last two characters

Push answer to stack

Postfix Expression Evaluation

5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

5

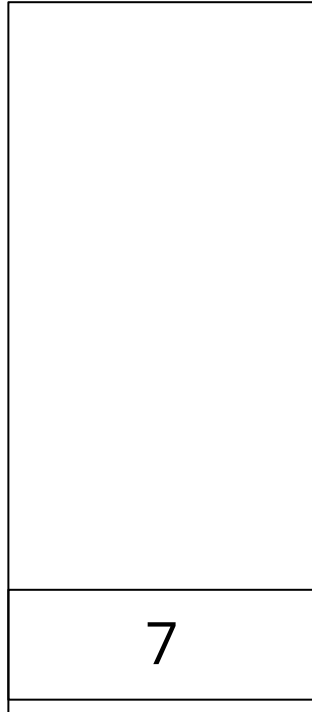
+

2

=

7

Postfix Expression Evaluation



5 2 + 3 1 - x

A red horizontal line is positioned under the '+' operator.

Possible solution:

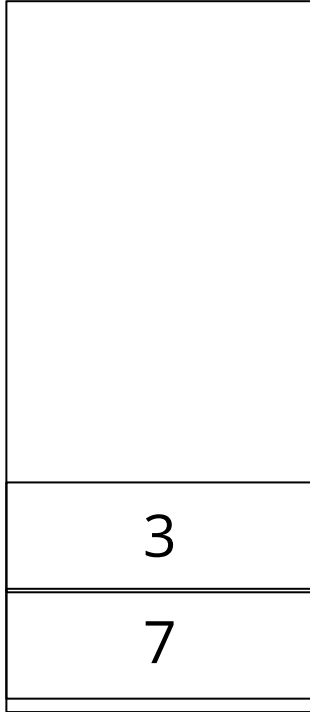
Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

$$\boxed{5} + \boxed{2} = \boxed{7}$$

Postfix Expression Evaluation



5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator

Pop last two characters from stack

Perform operation on last two characters

Push answer to stack

Postfix Expression Evaluation

1
3
7

5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

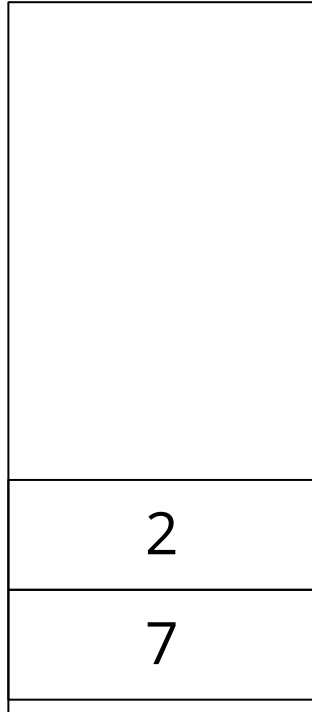
If character is an operator

Pop last two characters from stack

Perform operation on last two characters

Push answer to stack

Postfix Expression Evaluation



5 2 + 3 1 - x

—

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

3	-	1	=	2
---	---	---	---	---

Postfix Expression Evaluation

2
7

5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator

Pop last two characters from stack

Perform operation on last two characters

Push answer to stack

Postfix Expression Evaluation

5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

7

x

2

=

14

Postfix Expression Evaluation

5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

14

7

x

2

=

14

Postfix Expression Evaluation

14

5 2 + 3 1 - x

Possible solution:

Take characters one by one from the beginning

— If character is an operand
Push to stack

If character is an operator
Pop last two characters from stack
Perform operation on last two characters
Push answer to stack

Postfix Expression Evaluation - Another Example

Stack	Expression
 --- (empty)	3 2 + 4 * 5 1 - /
3 ---	2 + 4 * 5 1 - /
2 3 ---	+ 4 * 5 1 - /
5 ---	4 * 5 1 - /
4 5 ---	* 5 1 - /
20 ---	5 1 - /
5 20 ---	1 - /

1 5 20 ---	- /
4 20 ---	/
5 ---	(finished

Parenthesis Matching

Input: exp = "[()]{ }{[()()]()}"

Output: Balanced

Input: exp = "[()]"

Output: Not Balanced

Parenthesis Matching

Follow the steps mentioned below to implement the idea:

- Declare a character stack (say **temp**).
- Now traverse the string exp.
 - If the current character is a starting bracket ('(' or '{' or '[') then push it to stack.
 - If the current character is a closing bracket (')' or '}' or ']') then pop from the stack and if the popped character is the matching starting bracket then fine.
 - Else brackets are **Not Balanced**.
- After complete traversal, if some starting brackets are left in the stack then the expression is **Not balanced**, else **Balanced**.

Parenthesis Matching

Initially :



Step 1:



Parenthesis Matching

Step 2:

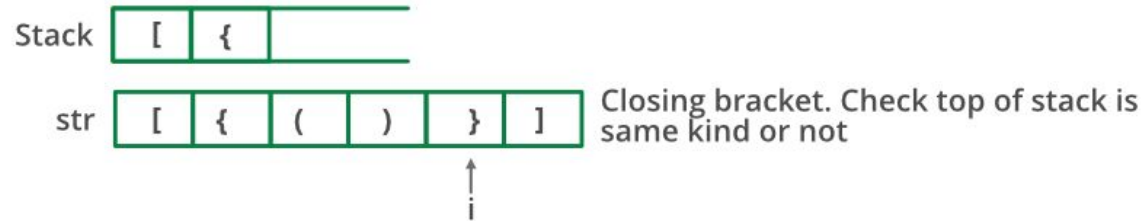


Step 3:

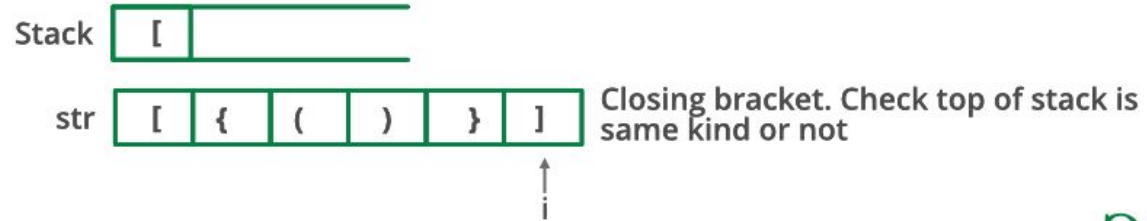


Parenthesis Matching

Step 4:



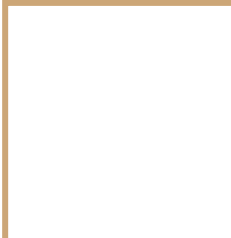
Step 5:



Stack Applications

- 5. Postfix Expression Evaluation
- 6. Parenthesis Matching

Try writing both programs yourselves.



Queue



Queue

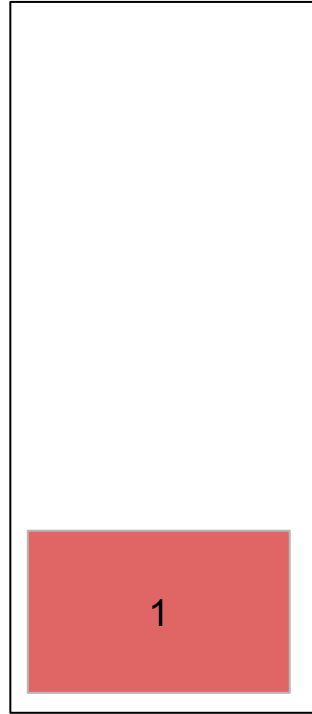
IN



OUT

Stack VS Queue

Stack ->



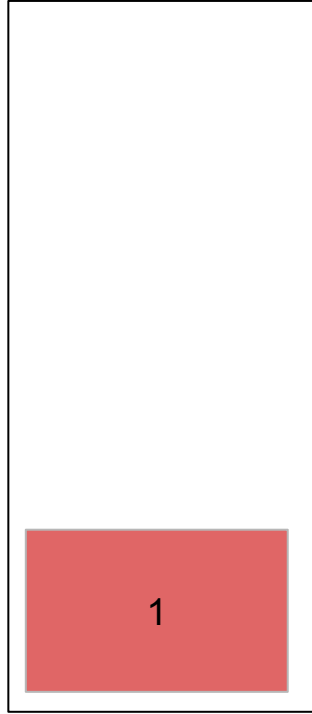
Last In First Out



Queue-> First In First Out

Push(1)

stack



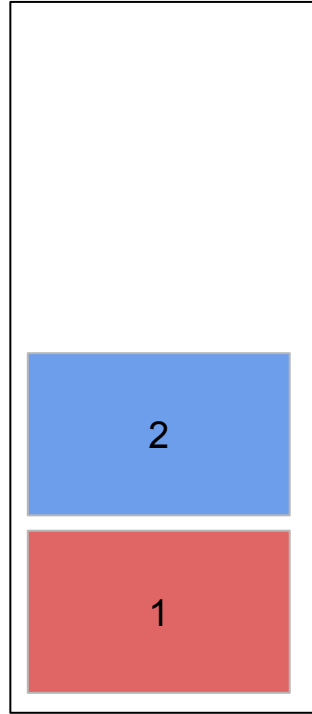
queue



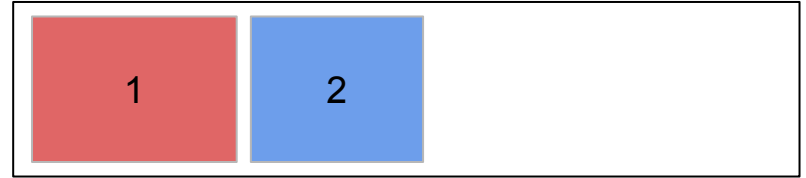
Enqueue(1)

Push(2)

stack



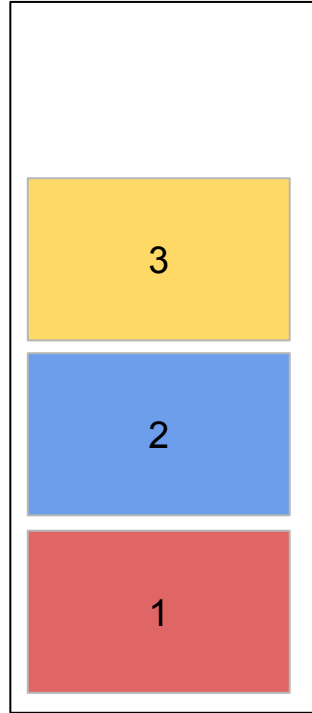
queue



Enqueue(2)

Push(3)

stack



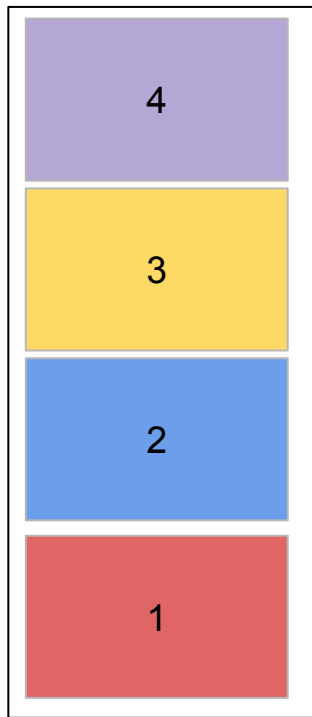
queue



Enqueue(3)

Push(4)

stack



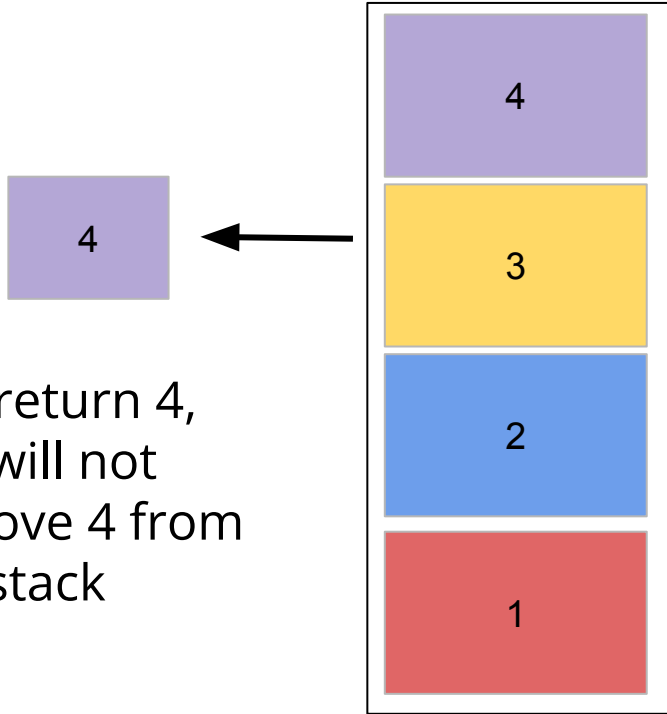
queue



Enqueue(4)

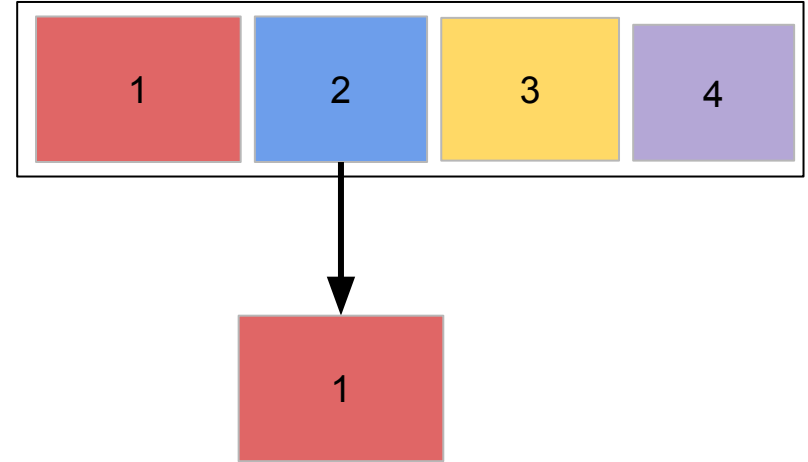
Peek

stack



Will return 4,
but will not
remove 4 from
the stack

queue



Will return 1,
but will not
remove 1 from
the queue

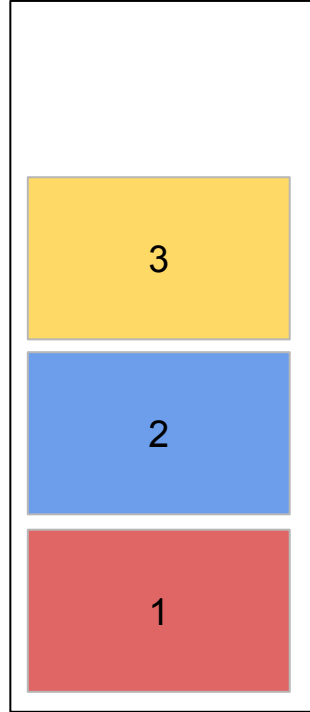
Peek

Pop



Will return 4 and
remove 4 from the
stack

stack



queue



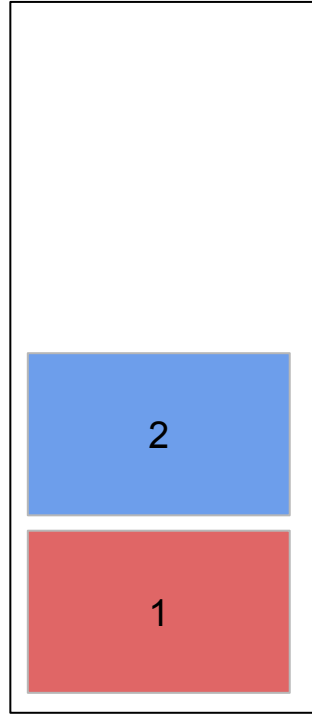
Will return 1 and remove 1
from the queue

Dequeue

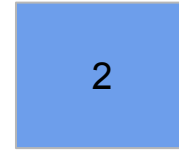
Pop



stack



queue

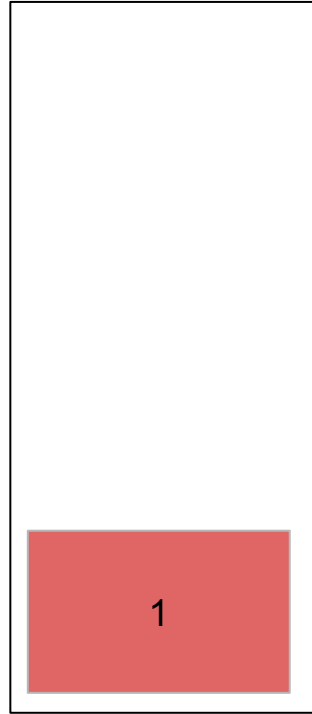


Dequeue

Pop



stack



queue

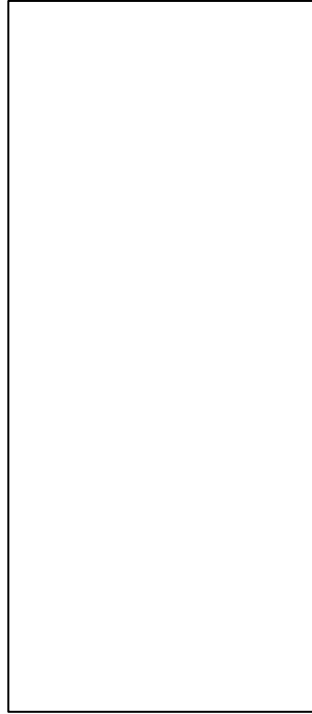


Dequeue

Pop



stack



queue



Dequeue

Queue

A queue has three main operations (like a stack). These operations are:

1. **Enqueue:** It stores a new object in the queue at the end. If the queue is full, this operation throws a `QueueOverflow` exception.
2. **Dequeue:** It removes the first object from the queue. If the queue is empty, this operation throws a `QueueUnderflow` exception.
3. **Peek:** This operation shows the first item in the queue. This operation does not remove any items from the queue. If the queue is empty, this operation throws a `QueueUnderflow` exception.

Queue

```
class Queue:
    def __init__(self, max_size):
        self.max_size = max_size
        self.queue = [None] * max_size
        self.front = 0
        self.rear = -1
        self.size = 0

    def enqueue(self, item):
        if self.size == self.max_size:
            print("Queue is full")
            return
        self.rear = (self.rear + 1) %
self.max_size
        self.queue[self.rear] = item
        self.size += 1
```

```
def dequeue(self):
    if self.size == 0:
        print("Queue is empty")
        return None
    item = self.queue[self.front]
    self.queue[self.front] = None
    self.front = (self.front + 1) % self.max_size
    self.size -= 1
    return item

def peek(self):
    if self.size == 0:
        print("Queue is empty")
        return None
    return self.queue[self.front]
```

Sidenote: A Refresher On Modular Operations

$$0\%4 = 0$$

$$1\%4 = 1$$

$$2\%4 = 2$$

$$3\%4 = 3$$

$$4\%4 = 0$$

$$5\%4 = 1$$

$$6\%4 = 2$$

Queue() #when `__init__` is called

size = 0
front=0

max_size=4



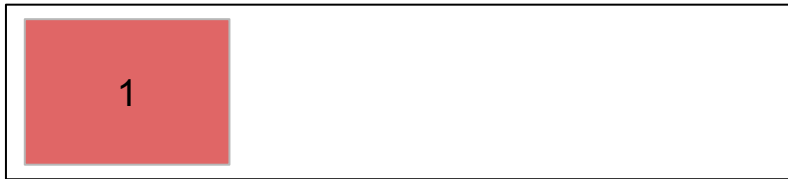
rear=-1

```
def __init__(self, max_size):  
    self.max_size = max_size  
    self.queue = [None] *  
max_size  
    self.front = 0  
    self.rear = -1  
    self.size = 0
```

Enqueue(1)

size = 1
front=0

max_size=4



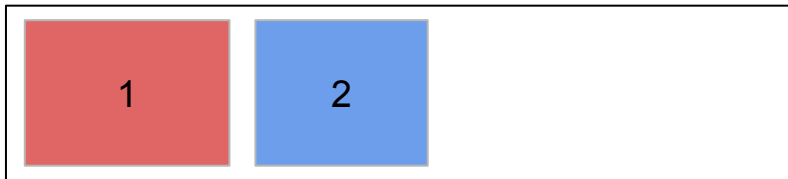
rear=0

```
def enqueue(self, item):  
    if self.size == self.max_size:  
        print("Queue is full")  
        return  
    self.rear = (self.rear + 1) %  
self.max_size  
    self.queue[self.rear] = item  
    self.size += 1
```

Enqueue(2)

size = 2
front=0

max_size=4



rear=1

```
def enqueue(self, item):  
    if self.size == self.max_size:  
        print("Queue is full")  
        return  
    self.rear = (self.rear + 1) %  
self.max_size  
    self.queue[self.rear] = item  
    self.size += 1
```

Enqueue(3)

size = 3
front=0

max_size=4



rear=2

```
def enqueue(self, item):  
    if self.size == self.max_size:  
        print("Queue is full")  
        return  
    self.rear = (self.rear + 1) %  
self.max_size  
    self.queue[self.rear] = item  
    self.size += 1
```


Enqueue(4)

size = 4
front=0

max_size=4



rear=3

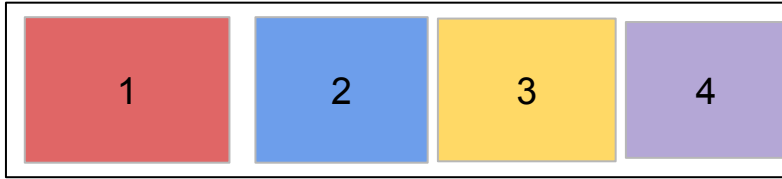
```
def enqueue(self, item):  
    if self.size == self.max_size:  
        print("Queue is full")  
        return  
    self.rear = (self.rear + 1) %  
self.max_size  
    self.queue[self.rear] = item  
    self.size += 1
```

Enqueue(5)

Queue Overflow Error!

size = 4
front=0

max_size=4



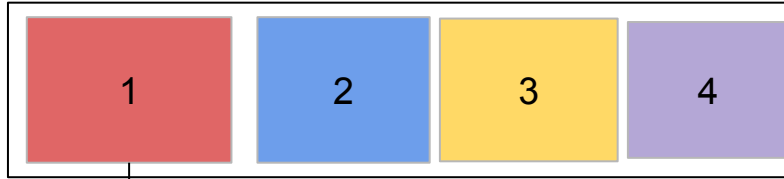
rear=3

```
def enqueue(self, item):  
    if self.size == self.max_size:  
        print("Queue is full")  
        return  
    self.rear = (self.rear + 1) %  
self.max_size  
    self.queue[self.rear] = item  
    self.size += 1
```

Peek

size = 4
front=0

max_size=4



rear=3



Will return 1 but will not
remove it

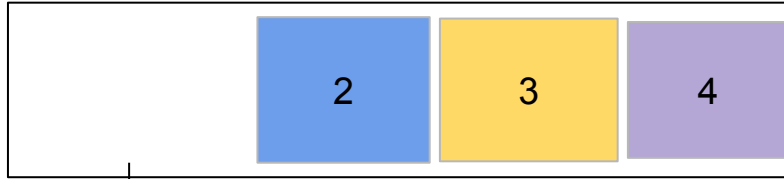
```
def peek(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    return self.queue[self.front]
```

Deque

size = 3

front=1

max_size=4



rear=3

Will return 1 and remove it

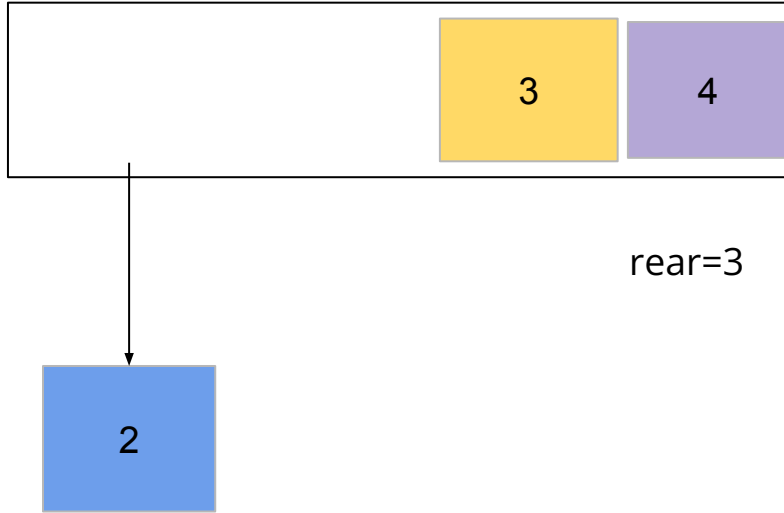
```
def dequeue(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    item = self.queue[self.front]  
    self.queue[self.front] = None  
    self.front = (self.front + 1) %  
self.max_size  
    self.size -= 1  
    return item
```

Deque

size = 2

front=2

max_size=4



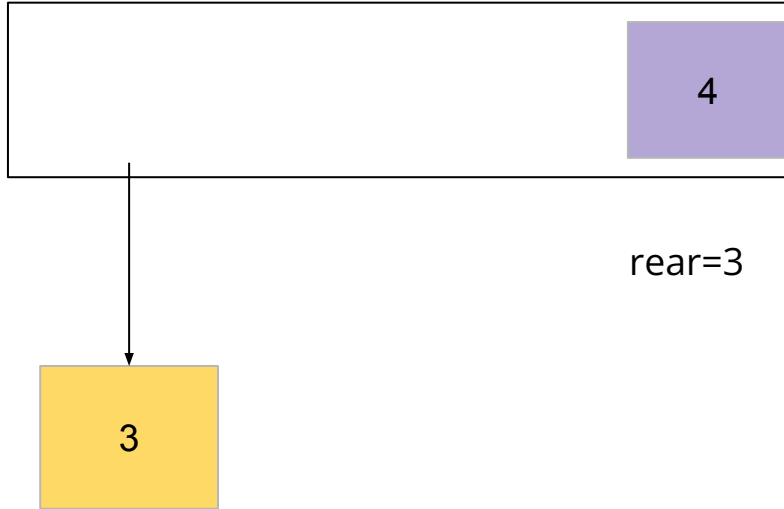
```
def dequeue(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    item = self.queue[self.front]  
    self.queue[self.front] = None  
    self.front = (self.front + 1) %  
self.max_size  
    self.size -= 1  
    return item
```

Deque

size = 1

max_size=4

front=3



```
def dequeue(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    item = self.queue[self.front]  
    self.queue[self.front] = None  
    self.front = (self.front + 1) %  
self.max_size  
    self.size -= 1  
    return item
```

Dequeue

size = 1

front=3

max_size=4



rear=3



What will happen if I dequeue now?
Think!

```
def dequeue(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    item = self.queue[self.front]  
    self.queue[self.front] = None  
    self.front = (self.front + 1) %  
self.max_size  
    self.size -= 1  
    return item
```

Deque

$$\text{front} \% \text{max_size} = 4 \% 4 = 0$$

size = 0

front=0



max_size=4

rear=3

```
def dequeue(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    item = self.queue[self.front]  
    self.queue[self.front] = None  
    self.front = (self.front + 1) %  
self.max_size  
    self.size -= 1  
    return item
```


Deque

Queue Underflow Error!

size = 0

front=0



max_size=4

rear=3

```
def dequeue(self):  
    if self.size == 0:  
        print("Queue is empty")  
        return None  
    item = self.queue[self.front]  
    self.queue[self.front] = None  
    self.front = (self.front + 1) %  
self.max_size  
    self.size -= 1  
    return item
```

size = 0

max_size=4

front=0



rear=3



What will happen if I enqueue something now? Think!

Enqueue(1)

$$\text{rear} \% \text{max_size} = 4 \% 4 = 0$$

size = 1
front=0

max_size=4



rear=0

```
def enqueue(self, item):  
    if self.size == self.max_size:  
        print("Queue is full")  
        return  
    self.rear = (self.rear + 1) %  
self.max_size  
    self.queue[self.rear] = item  
    self.size += 1
```

Queue - Linked List Based Implementation

```
1 class Queue:
2
3     def __init__(self):
4         self.front = None
5         self.back = None
6
7     def enqueue(self, elem):
8         if self.front == None: # This means first item
9             self.front = Node(elem, None)
10            self.back = self.front # Initially front and back are same
11        else:
12            n = Node(elem, None)
13            self.back.next = n # Inserting at the last
14            self.back = self.back.next # Moving back at the end
15
16    def dequeue(self):
17        if self.front == None: # For empty Queue
18            print("Queue Underflow Exception")
19        else:
20            dequeued_item = self.front.elem
21            self.front = self.front.next # Moving the front pointer
22            return dequeued_item # Returning the dequeued item
23
24    def peek(self):
25        if self.front == None: # For empty Queue
26            print("Queue Underflow Exception")
27        else:
28            return self.front.elem # Returning the front item
```

Not fixed sized, no need for modular operations

```
1 class Node:
2     def __init__(self, elem, next):
3         self.elem = elem
4         self.next = next
```

Simulations

Type 1

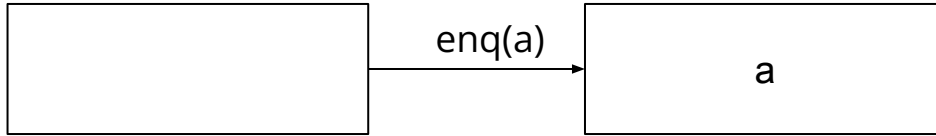
You need to do the following operations on a queue:

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue.

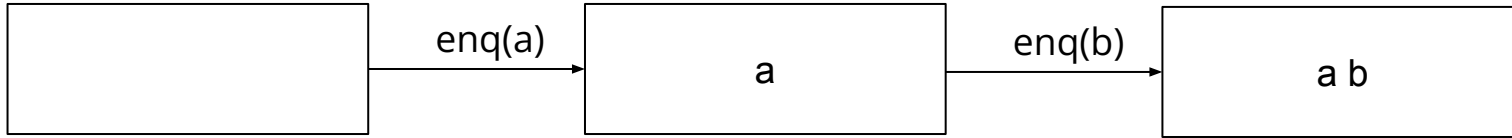
enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue



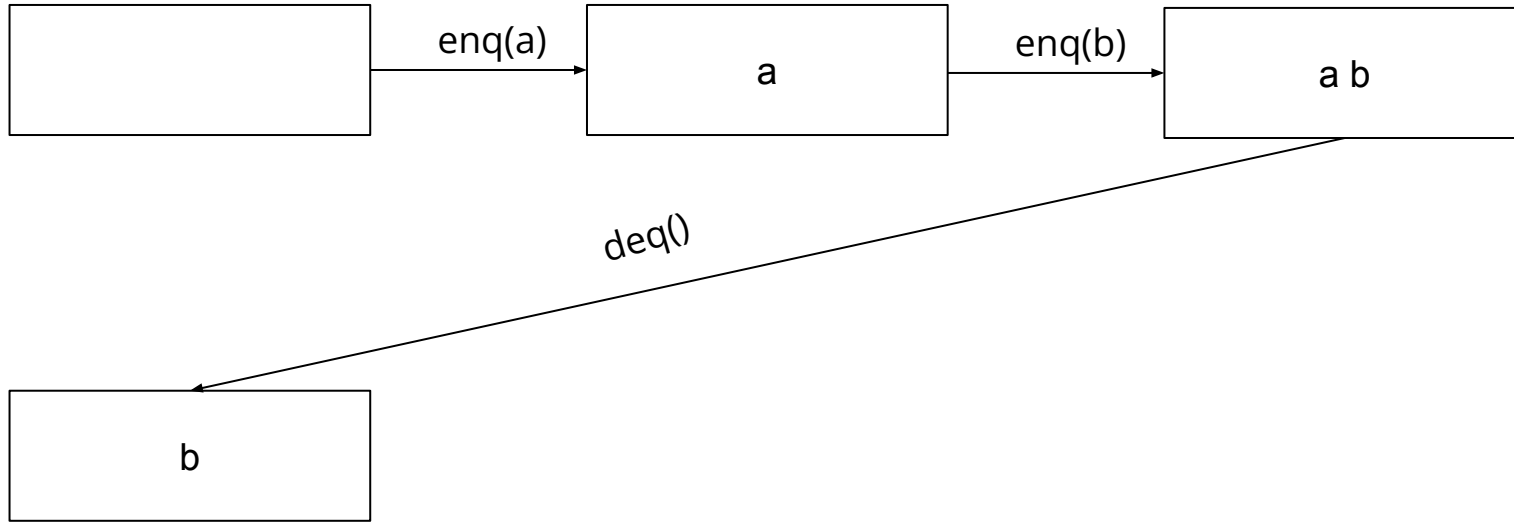
enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue



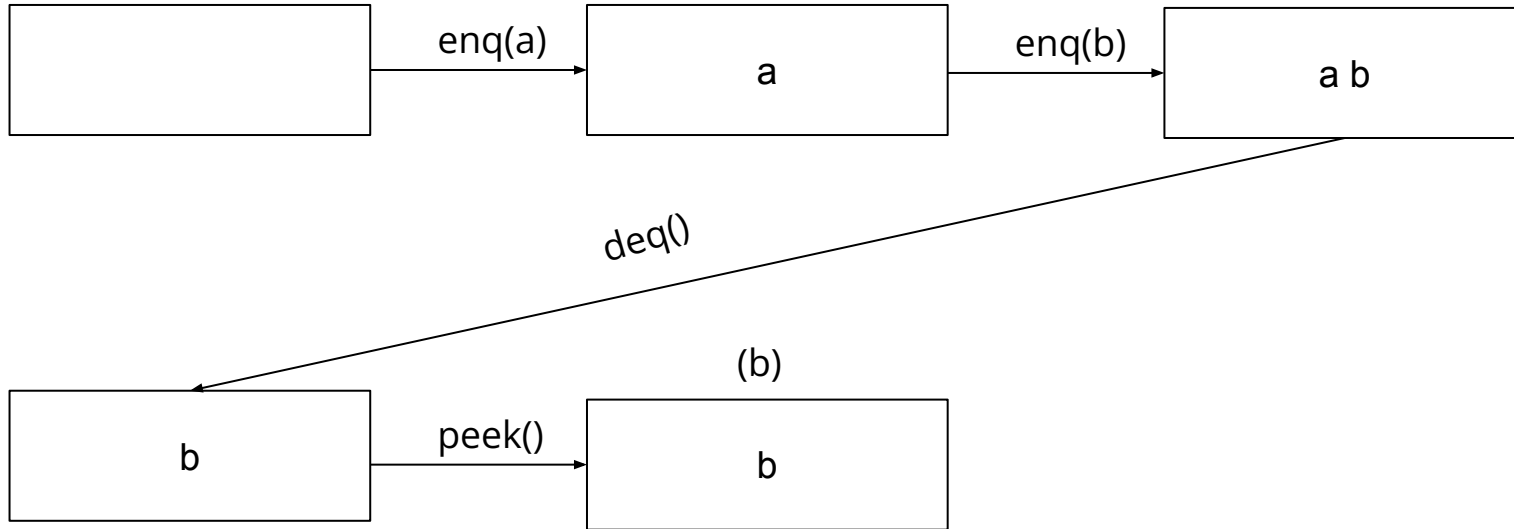
enqueue a, **enqueue b**, dequeue, peek, enqueue c, peek, dequeue



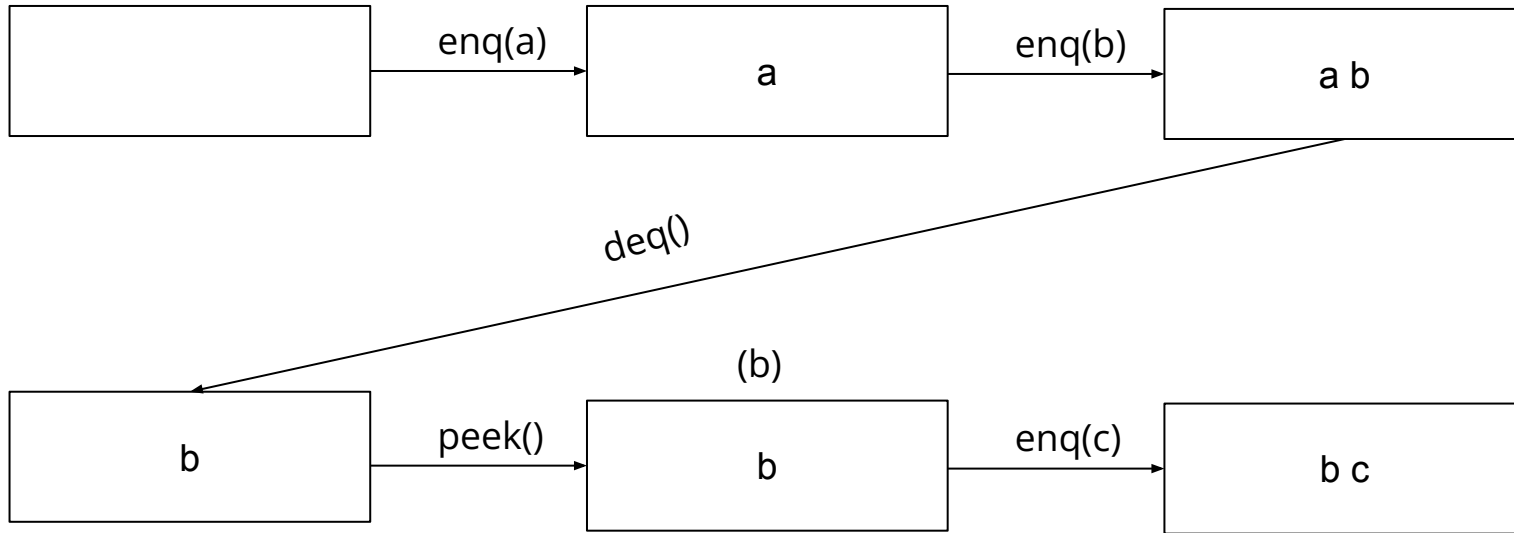
enqueue a, enqueue b, **dequeue**, peek, enqueue c, peek, dequeue



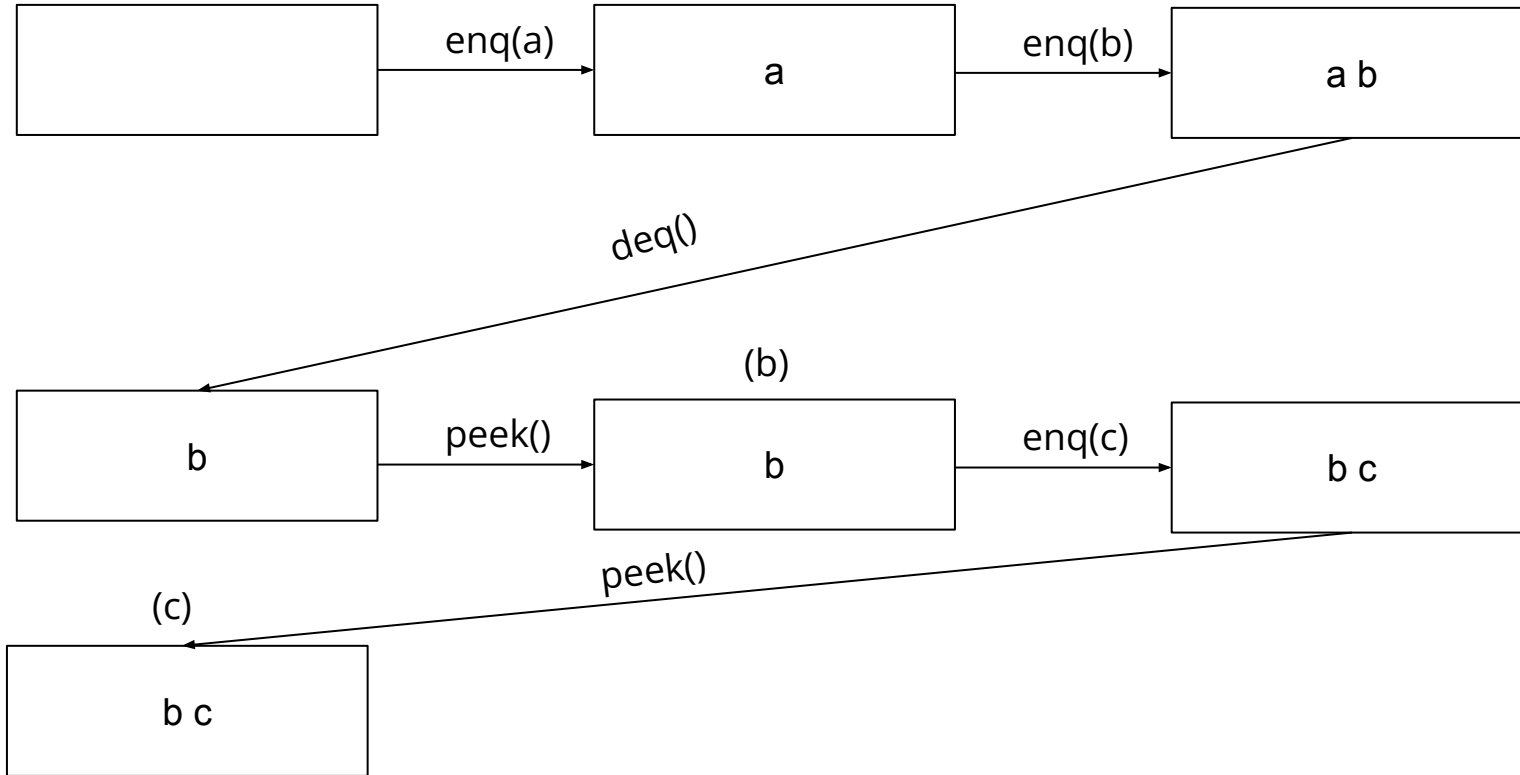
enqueue a, enqueue b, dequeue, **peek**, enqueue c, peek, dequeue



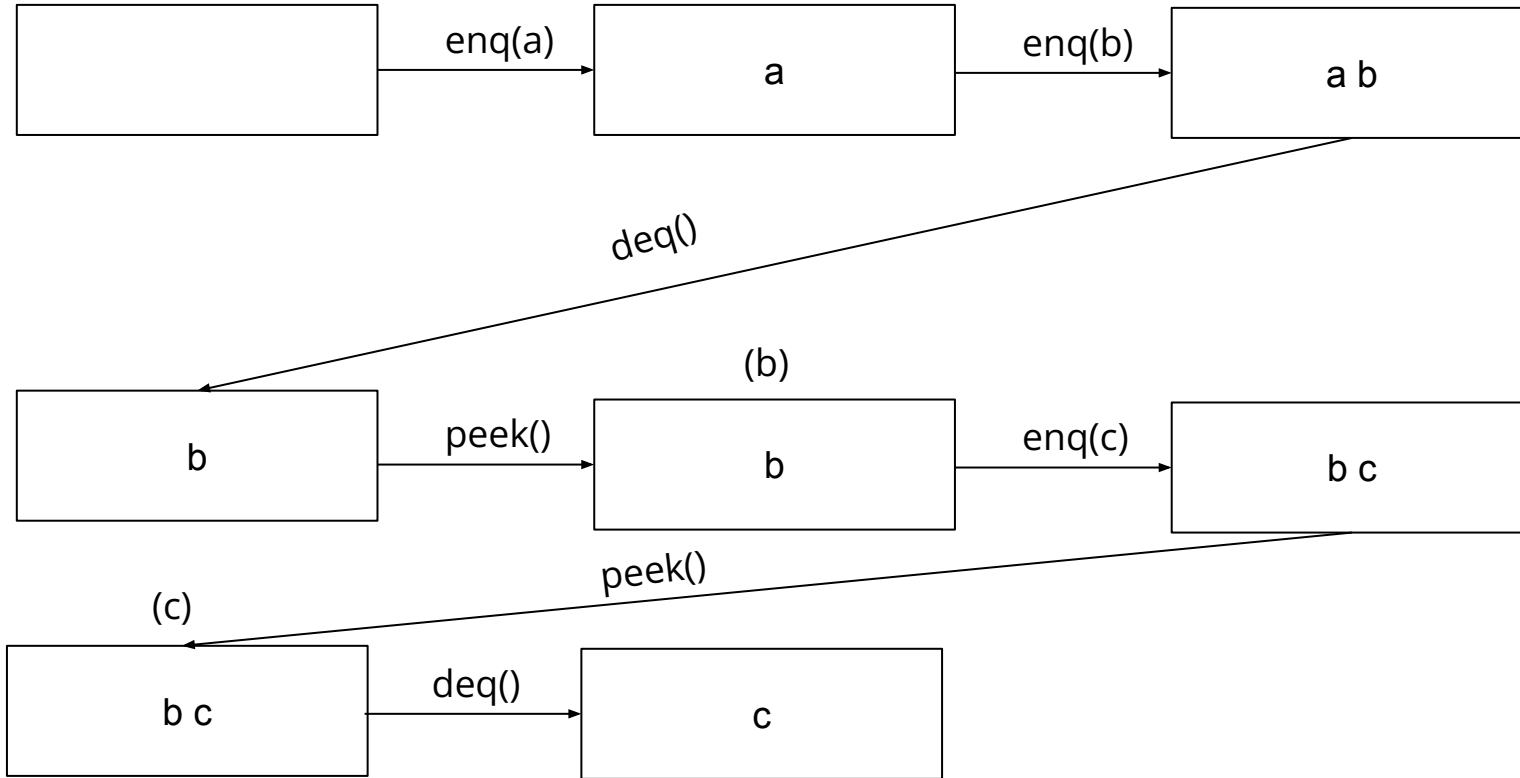
enqueue a, enqueue b, dequeue, peek, **enqueue c**, peek, dequeue



enqueue a, enqueue b, dequeue, peek, enqueue c, **peek**, dequeue

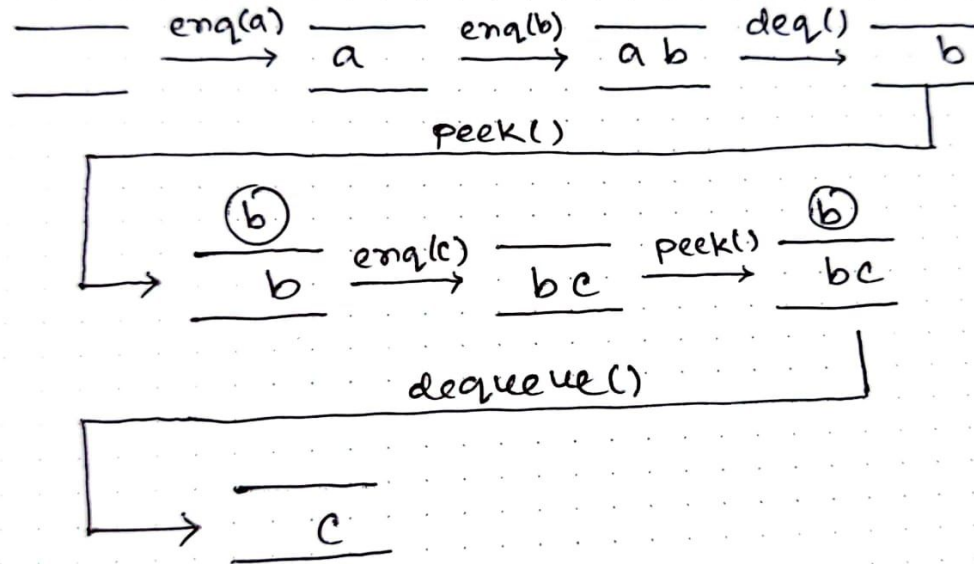


enqueue a, enqueue b, dequeue, peek, enqueue c, peek, **dequeue**



In exam, write your answer like this

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, **dequeue**



Simulations

Type 2

You need to perform the following operations on a array based queue where the length of array is 4 and starting index of front and back is 3.

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue.

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)				
enq(b)				
deq()				
peek()				
enq(c)				
peek()				
deq()				

front	rear
3	3

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)				
deq()				
peek()				
enq(c)				
peek()				
deq()				

front	rear
3	3
	0

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)	b	N	N	a
deq()				
peek()				
enq(c)				
peek()				
deq()				

front	rear
3	3
	0
	1

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)	b	N	N	a
deq()	b	N	N	N
peek()				
enq(c)				
peek()				
deq()				

front	rear
3	3
0	0
	1

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)	b	N	N	a
deq()	b	N	N	N
peek()	b	N	N	N
enq(c)				
peek()				
deq()				

front	rear
3	3
0	0
	1

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)	b	N	N	a
deq()	b	N	N	N
peek()	b	N	N	N
enq(c)	b	c	N	N
peek()				
deq()				

front	rear
3	3
0	0
	4
	2

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)	b	N	N	a
deq()	b	N	N	N
peek()	b	N	N	N
enq(c)	b	c	N	N
peek()	b	c	N	N
deq()				

front	rear
3	3
0	0
	4
	2

(N=None)

enqueue a, enqueue b, dequeue, peek, enqueue c, peek, dequeue

	0	1	2	3
	N	N	N	N
enq(a)	N	N	N	a
enq(b)	b	N	N	a
deq()	b	N	N	N
peek()	b	N	N	N
enq(c)	b	c	N	N
peek()	b	c	N	N
deq()	N	c	N	N

front	rear
3	3
0	0
1	4
	2

(N=None)

In exam, write your answer like this

	0	1	2	3	<u>front</u> <u>rear</u>	
	N	N	N	N	3	3
enq(a)	N	N	N	a	0	0
enq(b)	b	N	N	a	1	1
deq()	b	N	N	N		2
peek()	b	N	N	N		
enq(c)	b	c	N	N		
peek()	b	c	N	N		
dequeue()	N	c	N	N		

A Discussion on the Importance of Stacks and Queues

By now you well understand that larger complex data structures are made of smaller building block data structures. For example, multi-dimensional and circular arrays or vectors are made using linear arrays; lists are used to create sets and maps (which is called a dictionary in some languages and associated arrays in some other). However, as building block data structures, stacks and queues are unique in some sense.

A Discussion on the Importance of Stacks and Queues

For example, a set or a map that you create from a linked list provides additional features over their building block component that is the list. However, stacks and queues are strictly restricted forms of linked list (in rare cases, where you know the maximum number of elements you will put in them, you can use arrays for stacks and queues also). In other words, they do not provide any new features that a linked list does not already have. So it is a natural question why stacks and queues are considered fundamental data structures.

A Discussion on the Importance of Stacks and Queues

One reason for their importance is that scenarios where you need a linked list behave like a stack or queue is very frequent. For example, you need stacks for language parsing, expression evaluation, function calls, efficient graph traversals, etc. Queues are similarly important for resource scheduling, time sharing of CPU among candidate processes, message routing in network switches and routers, and efficient graph traversal.

A Discussion on the Importance of Stacks and Queues

Another central reason is that you often do not want to give access to the underlying list data structure that forms a stack or queue to the code that requested an insert or a removal from the stack/queue. If access to the list is provided then the code becomes more insecure. To consider this case, imagine direct access to the function call stack is possible. Then a program can manipulate the stack and jump from the current function to a much earlier function, instead of the immediate predecessor function that called it. Why this might be a problem? Well, this would not be a problem if all the functions in the stack belong to your own program. However, in real-world, library functions, operating system's service functions interleave with user program's functions in the stack. Unprotected jumps to those functions can crash the system.

A Discussion on the Importance of Stacks and Queues

A third reason to have stack and queues as restricted form of linked lists (or arrays) is that their restricted functions provide smaller code that can fit in very short memory or can even be implemented inside hardware devices directly. This feature is particularly important when you need fast response, for example, in routers and switches. In fact, array based stack and queue implementations are most suited in cases like this.